

Intro

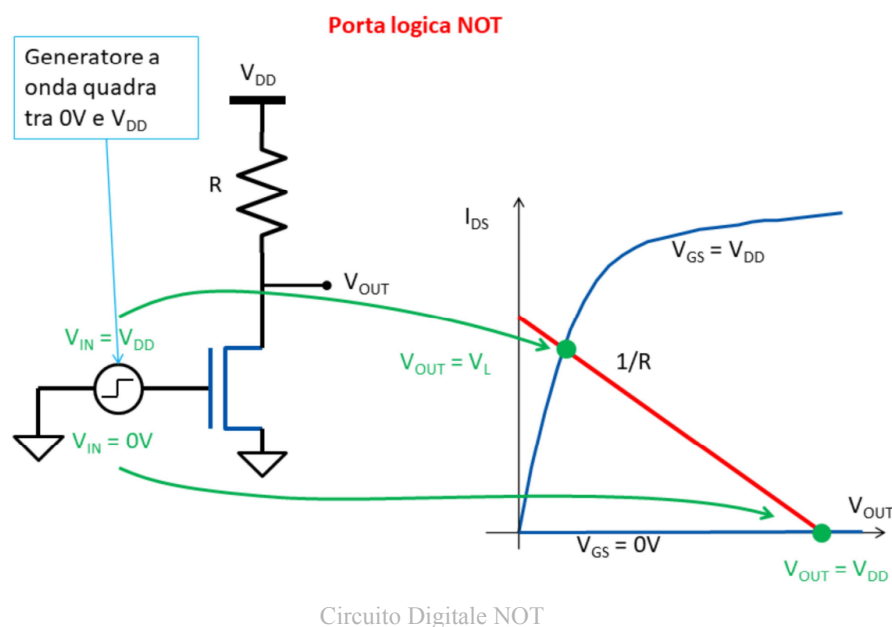
I circuiti integrati sono alla base dei computer

Legge di Moore

Le prestazioni delle cpu raddoppiano ogni 18 mesi.

Un segnale è un supporto fisico che contiene un'informazione che viene trasferita da una sorgente ad un utilizzatore. In questo corso si studieranno i **segnali elettrici** e **digitali**. Per mettere in relazione i due valori binari ai valori analogici si definisce una massa $0V = V_L = 0[b]$ e un potenziale $V_{DD} = V_H = 1[b]$.

Ci sono delle porte logiche elementari che costruiremo con circuiti elettronici. Ma come si crea un circuito binario che funziona con valori analogici?

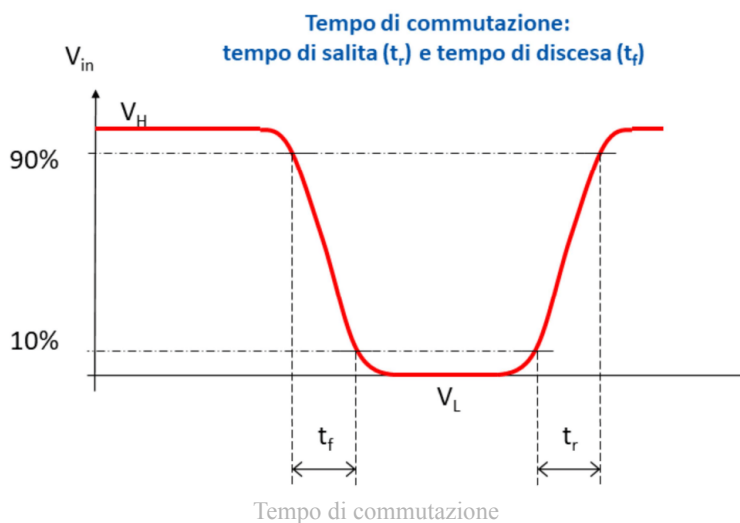


Essendo un'onda quadra (supponiamo ideale) il gate assumerà potenziale 0 o V_{DD} . Se l'ingresso è nullo $V_{GS} < V_{th}$ e quindi il transistor è spento: $V_{out} = V_{DD}$. Invece, con un valore di V_{DD} corretto si ha il MOS in saturazione e quindi si crea una corrente V_{out} sarà minore ≈ 0 . Basta guardare la retta di carico (rossa). Chiaramente più il voltaggio è alto, più l'uscita sarà piccola. I cambiamenti avvengono **in modo continuo**.

Parametri Dinamici

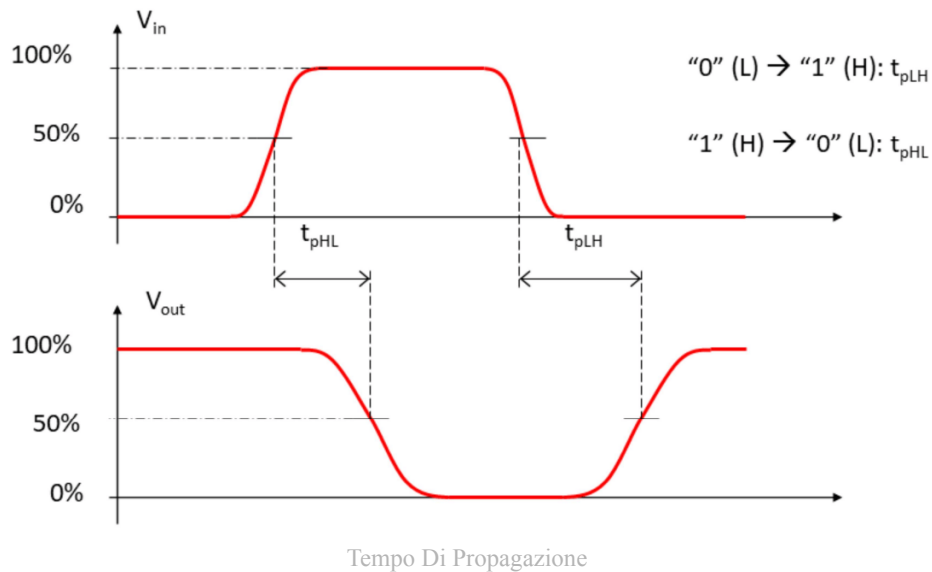
Tempo di commutazione:

Questo è il tempo che l'uscita ci mette ad arrivare da una differenza dello stato corrente 10% a una differenza del 10% allo stato successivo.



Tempo di propagazione:

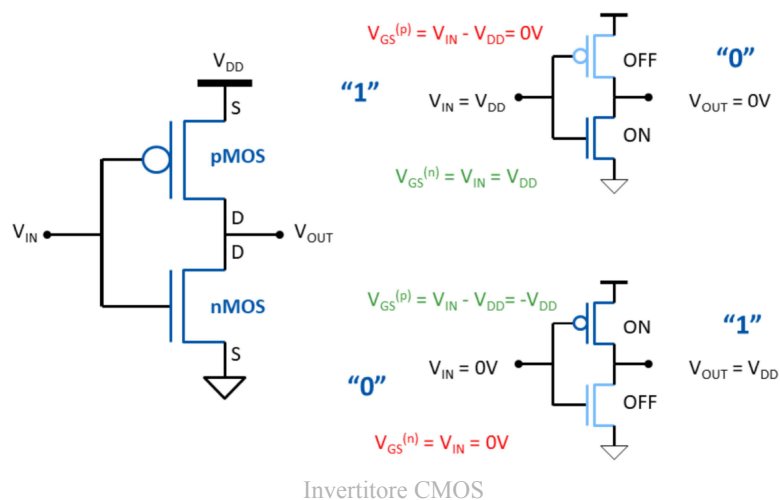
Questo indica il tempo per far commutare l'uscita rispetto all'input. Si usano le sigle LH o HL per definire il tempo per andare da 0 a 1 e rispettivamente da 1 a 0. **Si prende come riferimento il livello intermedio.** Infine si può definire il **tempo di propagazione** t_p come la media dei due tempo di ritardo



Invertitore CMOS Reale

Invece che Una resistenza mettiamo un pMOS (comportamento opposto al pMOS).

- $V_{in} = 0$: nMOS spento e pMOS acceso
- $V_{in} = 1$: nMOS acceso e pMOS spento



1) MOSFET Transistor

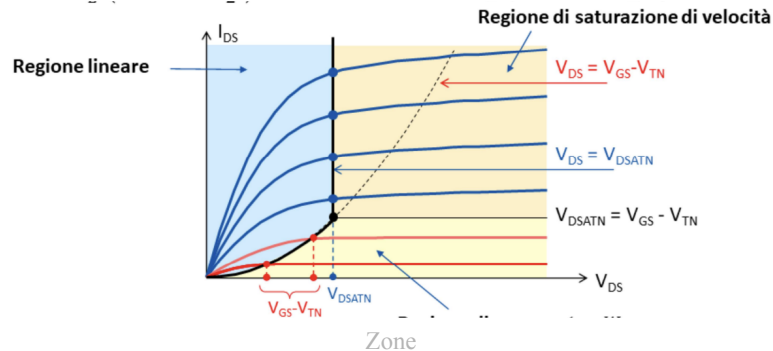
Accenni di Teoria

W e L indicano letteralmente la larghezza e lunghezza del canale del gate.
 NMOS ha s+ al source, s- al drain e p+ alla base. La struttura verge su p-
 PMOS ha p invertito con n

- **Cut off**

Qui abbiamo $I_{DS} = 0$ ma in realtà leggermente c'è. Condizione è che $0 < V_{GS} < V_{tn}$

Da qui in poi $V_{GS} > V_{tn}$ la regione dipende da quanto verrà ripotato più in basso. Accendendosi un MOS entrerà in una delle 3 zone. l'ordine di queste cambia. Tenzialmente inizia con la zone lineare, poi andrà a saturarsi (canale corto) o in pinch off o in velocità

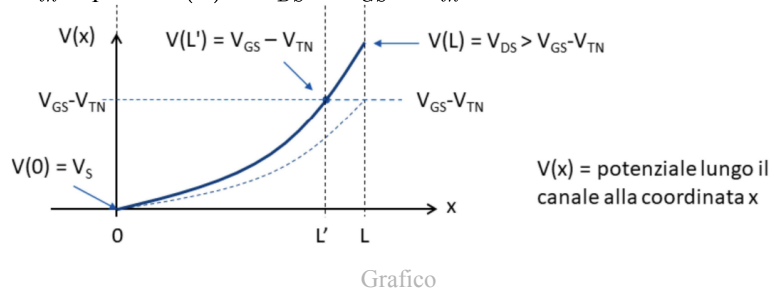


l'ordine è più evidente. satura in velocità se $V_{DSATN} > V_{GS} - V_{tn}$

Il substrato di NMOS deve essere connesso al potenziale più basso, il PMOS invece al più alto

1. modulazione lunghezza di canale

V_{DS} va a spostare il punto di pinchoff. L'estensione massima impone (pinch off al drain)
 $V(L) = V_{DS} = V_{GS} - V_{tn}$. Allontanando il punto di pinchoff è possibile avere un punto
 $V(L' < L) = V_{GS} - V_{tn}$ e quindi $V(L) = V_{DS} > V_{GS} - V_{tn}$



2. saturazione di velocità

Il nome viene letteralmente dal fatto che all'aumentare di V_{DS} si raggiunge l'intensità critica del campo lungo il canale e le cariche andranno a velocità costante: non dipendono più da V_{DS}

Recap dei modelli a canale corto dei n/p MOSFET

- **NMOS**

Spento se $V_{GS} < V_{tn}$

Quando è acceso ci sono 3 zone che si scelgono in base al $\min\{V_{DS}, V_{GS} - V_{TN}, V_{DSATN}\}$

- V_{DS} : Regione Lineare
- $V_{GS} - V_{TN}$: Regione Saturazione (Pinch Off) $V_{DS} = V_{GS} - V_{TN}$
- V_{DSATN} : Regione Saturazione Velocità

$$I_{DS} = k'_n \frac{W_n}{L_n} \left(V_{GS} - V_{TN} - \frac{V_{MIN}}{2} \right) V_{MIN} \cdot (1 + \lambda_n V_{DS})$$

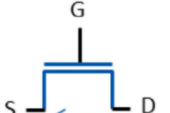
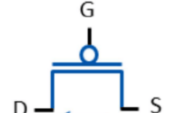
- **PMOS**

Spento se $V_{GS} > V_{TP}$

Quando è acceso ci sono 3 zone che si scelgono in base al $\max\{V_{DS}, V_{GS} - V_{TP}, V_{DSATP}\}$

- V_{DS} : Regione Lineare
- $V_{GS} - V_{TP}$: Regione Saturazione (Pinch Off) $V_{DS} = V_{GS} - V_{TP}$
- V_{DSATP} : Regione Saturazione Velocità

$$I_{DS} = k'_p \frac{W_p}{L_p} \left(V_{GS} - V_{TP} - \frac{V_{MAX}}{2} \right) V_{MAX} \cdot (1 + \lambda_p V_{DS})$$

Convenzione dei segni di tensioni e correnti			
	substrato connesso al potenziale più basso (tipicamente la massa) $V_{DS} > 0 \quad V_{SB} \geq 0$		substrato connesso al potenziale più alto (tipicamente l'alimentazione) $V_{DS} < 0 \quad V_{BS} \geq 0$
Convenzione dei segni dei parametri			
$k'_n > 0 \quad \lambda_n > 0 \quad V_{DSATN} > 0 \quad \gamma_n > 0 \quad \Phi_n > 0$		$k'_p > 0 \quad \lambda_p < 0 \quad V_{DSATP} < 0 \quad \gamma_p > 0 \quad \Phi_p > 0$	

Recappono

Effetto Body

Inoltre è anche importante parlare **dell'effetto body**. Questo avviene quando tra source e substrato (B) non c'è corto a massa, bensì una differenza di potenziale $V_{SB} > 0$. In questo caso la tensione di soglia dipende da quella SB secondo

$$V_{tn} = V_{tn0} + \gamma(\sqrt{V_{SB} + 2\varphi_n} - \sqrt{2\varphi_n})$$

$$V_{tp} = V_{tp0} - \gamma(\sqrt{V_{BS} + 2\varphi_n} - \sqrt{2\varphi_n})$$

dove γ, φ_n sono dati dal processo di fabbricazione. (V_{t0} è definito senza effetto body). Questo fenomeno va ad aumentare il modulo di V_{tn}

Nei MOSFET discreti SB è cortocircuitato $\implies$ niente effetto body

Remark (Come Calcolare Iterativamente V_S).

- **NMOS**

1. $V_{FH} = V_G - V_{tn0}$ (recall $V_S = V_{FH}$ nei NMOS, usually $V_G = V_{DD}$)
2. $V_{tn} \rightarrow V_{tn0} + \gamma(\sqrt{2\varphi_n + V_{FH}} - \sqrt{2\varphi_n}) \implies V_{FH} \rightarrow V_G - V_{tn}$ ripetere fino a convergenza (recall usually $V_B = 0$)

- **PMOS**

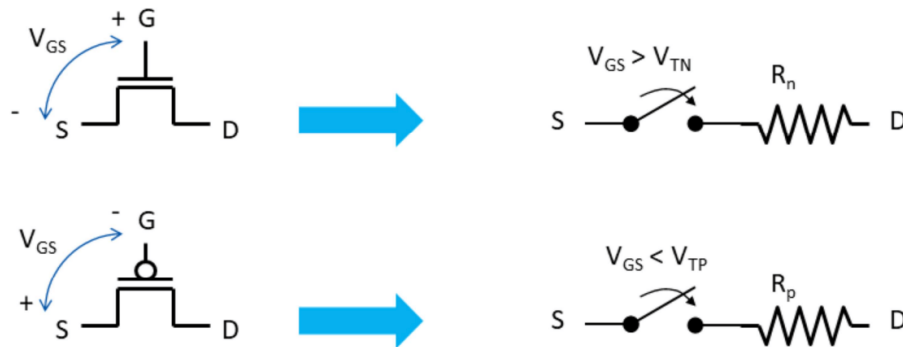
1. $V_{FL} = V_G - V_{tp0}$ (recall $V_S = V_{FL}$ nei PMOS, usually $V_G = 0$)
2. $V_{tp} \rightarrow V_{tp0} - \gamma(\sqrt{2\varphi_n + V_B - V_{FL}} - \sqrt{2\varphi_n}) \implies V_{FL} \rightarrow V_G - V_{tp}$ ripetere fino a convergenza (recall usually $V_B = V_{DD}$)

Fattore di Forma

Definiamo il fattore di forma come $Z = \frac{W}{L}$. Da qui, possiamo cambiare la definizione della corrente di saturazione $I_{DSAT} = Z I_{DSAT0}$ con $I_{DSAT0} = I_{DS}/Z$. (I_{DSAT0} è definito con $Z = 1$ e dato che è in sat di velocità $V_{GS} = \pm V_{DD}$)

MOSFET Come Interruttore e Resistenza Equivalente

Modelliamo i MOSFET come degli interruttori, Il modello prevede una resistenza data da quella interna del transistor.



Modello Transistor Interruttore

Dato che la corrente in un transistor non è lineare, nemmeno R_{DS} sarà costante. Non si può usare l'approssimazione lineare ai piccoli segnali dato che il segnale assume solo valori binari.

Useremo un **valore medio di resistenza** con le seguenti ipotesi:

- Il MOSFET è alimentato tra massa e V_{DD}
- V_{GS} riesce a far saturare il MOSFET
- $0 < V_{DS} < V_{DD}$

$$R_0 = \frac{2}{V_{DD}} \int_{V_{DD}/2}^{V_{DD}} \frac{V_{DS}}{I_{DSAT}} dV_{DS} = \frac{3}{4} \frac{V_{DD}}{I_{DSAT0}} \rightarrow R = \frac{R_0}{Z}$$

Questi risultati tengono per entrambi (n/p) i MOS. (R_0 definito con $Z = 1$).

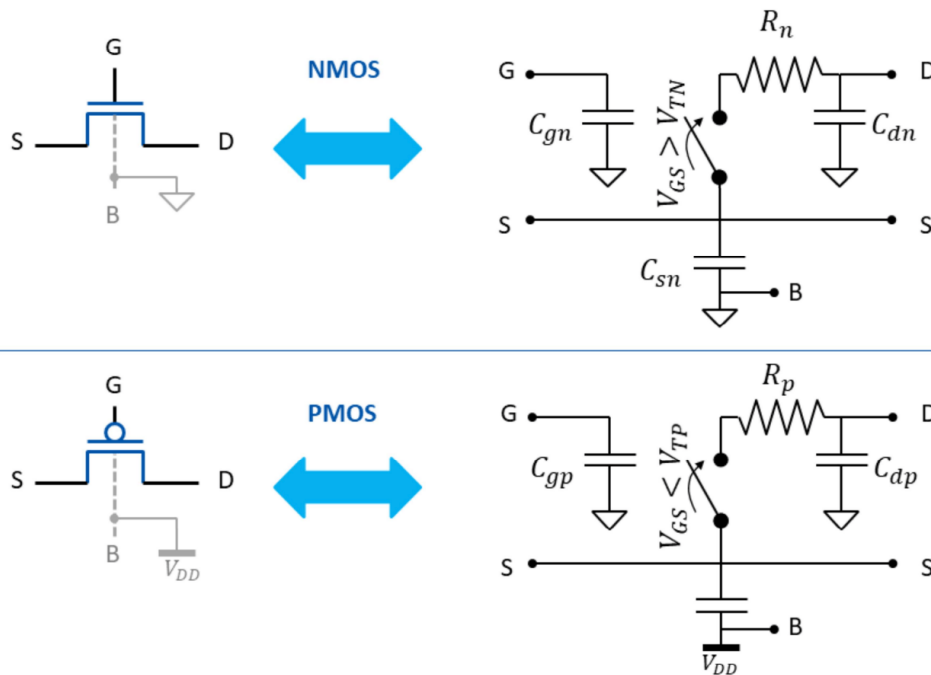
Come sono i valori tipici in relazione tra loro?

$$\begin{aligned} k'_n &= 3k'_p \\ V_{DSATN} &= \frac{2}{3} |V_{DSATP}| \\ V_{TN} &= -V_{TP} \end{aligned}$$

Quindi il rapporto tra le due resistenze, con V_{DD} sensibilmente grande è: $\frac{R_{n0}}{R_{p0}} \approx 2$.

Capacità Parassite in un MOS

I diodi che compongono il substrato del transistor che creano capacità parassite. Si possono definire delle **capacità cumulative di gate, source e drain**.



Modello Interruttore Con Capacità Parassite

Con L costante in tutti i transistor si possono fare le seguenti generalizzazioni. Definendo C_{g0}, C_{s0}, C_{d0} i valori per un MOS con $Z = 1$, allora la formula sarà valida per tutti i 3 casi

$$C = C_0 \cdot Z$$

Trasmettere un Segnale Digitale

^6df7d8

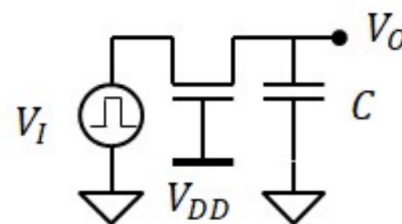
Un NMOS trasmette bene il valore logico basso, ma male quello alto

Un NMOS trasmette (a regime) correttamente i valori $\in [0, V_{max}]$. Il valore di V_{max} è il valore in cui il NMOS si spegne, ed è anche affetto da effetto body. Si trova risolvendo la seguente equazione:

$$V_{out} = V_{DD} - V_{TN} - \gamma(\sqrt{V_{out} + 2\phi} - \sqrt{2\phi})$$

Infine la trasmissione può essere riassunta così:

$$V_{out} = \begin{cases} V_{in}, & V_{in} \in [0, V_{max}] \\ V_{max}, & V_{in} > V_{max} \end{cases}$$

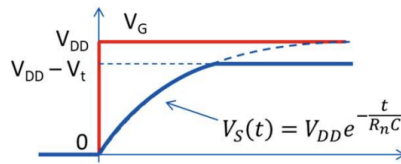


NMOS con C esplicitata

La caratteristica di tempo è data da $\tau = R_n \cdot C_{dn}$

$$\text{Carica: } V_{out}(t) = \min\{V_{DD}(1 - e^{-\frac{t}{\tau}}, V_{OH})\}$$

$$\text{Scarica: } V_{out}(t) = V_{OH} \cdot e^{-\frac{t}{\tau}}$$



Esempio Funzione a Tratti di Carica NMOS

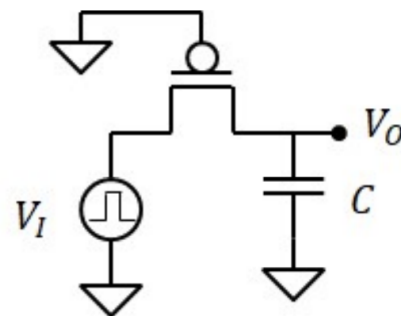
Un PMOS trasmette bene un valore logico alto, ma non uno basso

Un PMOS trasmette (a regime) correttamente i valori $\in [V_{min}, \infty]$. Il valore di V_{min} si trova risolvendo la seguente equazione:

$$V_{out} = V_{DD} - V_{TP} + \gamma(\sqrt{V_{DD} - V_{out} + 2\phi} - \sqrt{2\phi})$$

Infine la trasmissione può essere riassunta così:

$$V_{out} = \begin{cases} V_{min}, & V_{in} < V_{min} \\ V_{in}, & V_{in} \in [V_{min}, \infty] \end{cases}$$



PMOS con C esplicitata

La caratteristica di tempo è data da $\tau = R_p \cdot C_{dp}$

$$\text{Carica: } V_{out}(t) = V_{OL}(1 - e^{-\frac{t}{\tau}})$$

$$\text{Scarica: } V_{out}(t) = \max\{V_{DD} \cdot e^{-\frac{t}{\tau}}, V_{OL}\}$$

Un parallelo di NMOS e PMOS è in grado di trasmettere perfettamente valori alti e bassi, in quanto non si spengono mai contemporaneamente.

Reti di MOSFET

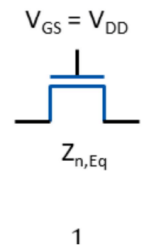
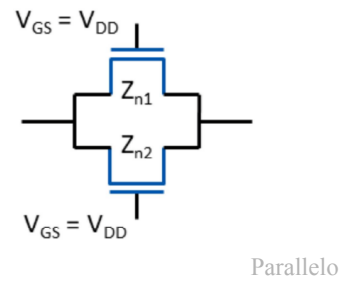
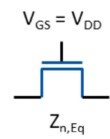
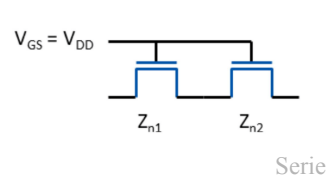
Ora cerchiamo di capire come definire un MOSET equivalente ad una rete:

Una serie di MOSFET è equivalente ad una serie di resistenze. Dato che il MOSFET è caratterizzato da $R_1 = R_0/Z_1$ il fattore di forma equivalente sarà un parallelo:

$$\frac{1}{Z_{n,eq}} = \frac{1}{Z_1} + \frac{1}{Z_2}$$

Discorso analogo per un parallelo di MOSFET. Qui il circuito equivale ad un parallelo di resistenze e quindi

$$Z_{n,eq} = Z_1 + Z_2$$



2) Invertitore CMOS

Come visto in precedenza l'invertitore CMOS si comporta come un NOT gate utilizzando un PMOS e un NMOS per trasmettere i simboli come visto in [trasmettere un segnale digitale](#).

- Ingresso basso: NMOS OFF, PMOS ON: l'uscita è V_{DD}

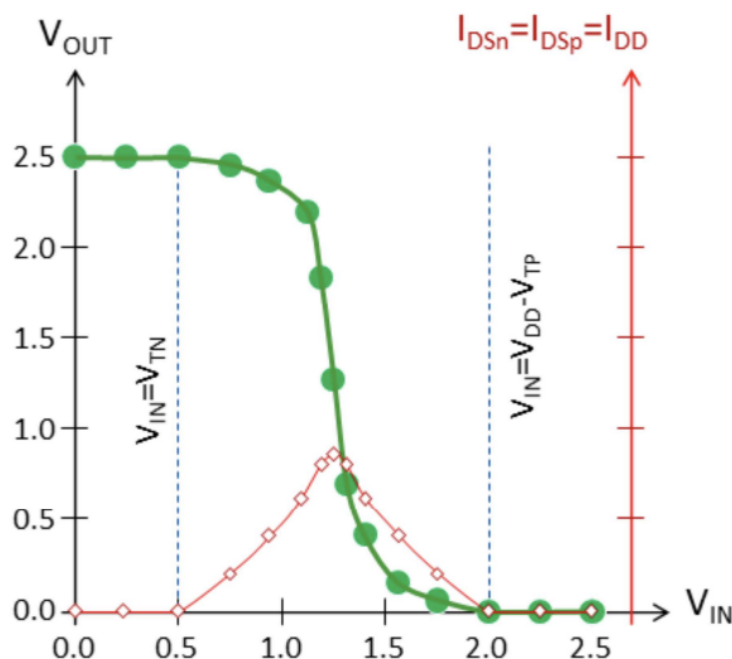
- Ingresso alto: NMOS ON, PMOS OFF: uscita $0V$

In ogni caso UNO e solo uno dei MOSFET è ON.

Il circuito è **statico** perchè c'è sempre un cammino a bassa impedenza che connette ad un potenziale noto. è **complementare** in quanto l'uscita non è mai connessa ad entrambi i potenziali

Caratteristiche di Trasferimento Ingresso-Uscita

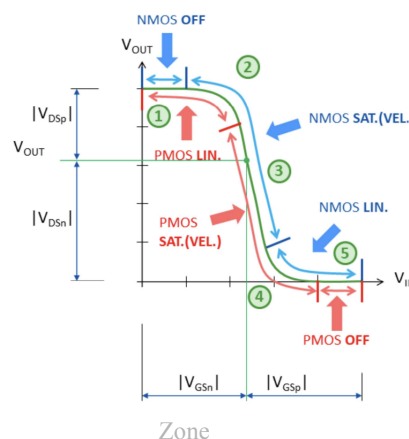
Si intende un grafico con V_{out} in funzione di V_{in} supponendo che la corrente per i due transistor sia uguale. Ai nostri scopi basta una curva approssimativa:



Esempio con $V_{DD}=2.5V$ e $V_{tn}=0.5V$

Si può notare come la corrente sia nulla con almeno uno dei MOS spenti. Da qui si possono individuare per ogni transistor le zone di OFF, Saturazione, Saturazione di velocità e Lineare. Da qui si possono individuare 5 regioni principali:

1. NMOS OFF, PMOS LIN, $V_{out} = V_{DD}$
2. NMOS SAT, PMOS LIN, ingresso medio-basso, uscita alta
3. Entrambi SAT, parte centrale, ingresso e uscita medi
4. NMOS LIN, PMOS SAT, ingresso medio-alto, uscita basso
5. NMOS LIN, PMOS OFF, $V_{out} = 0V$



Soglia Di Commutazione

Il punto operativo della VTC in cui $V_{out} = V_{in}$ è chiamato **soglia di commutazione**.

In questo istante i due MOS sono in saturazione di velocità, pertanto saranno accesi con la stessa corrente passante in entrambi. Quindi il potenziale $V_G = V_M$ ed eguagliando le due correnti si ottiene la formula analitica

$$V_M = \frac{V_{TN} + \frac{V_{DSATN}}{2} + r(V_{DD} + V_{TP} + \frac{V_{DSATP}}{2})}{1 + r}$$

con

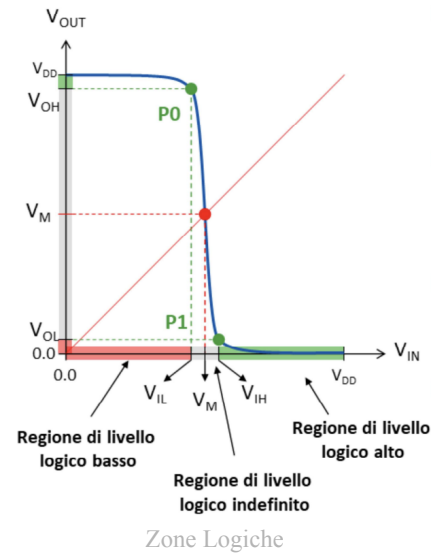
$$r = -\frac{k'_p Z_p V_{DSATP}}{k'_n Z_n V_{DSATN}}$$

Inoltre possiamo definire i punti P0 e P1 in cui $V_{in} = V_{il}, V_{ih}$ che dividono le region di di livello logico con la zona centrale indefinita

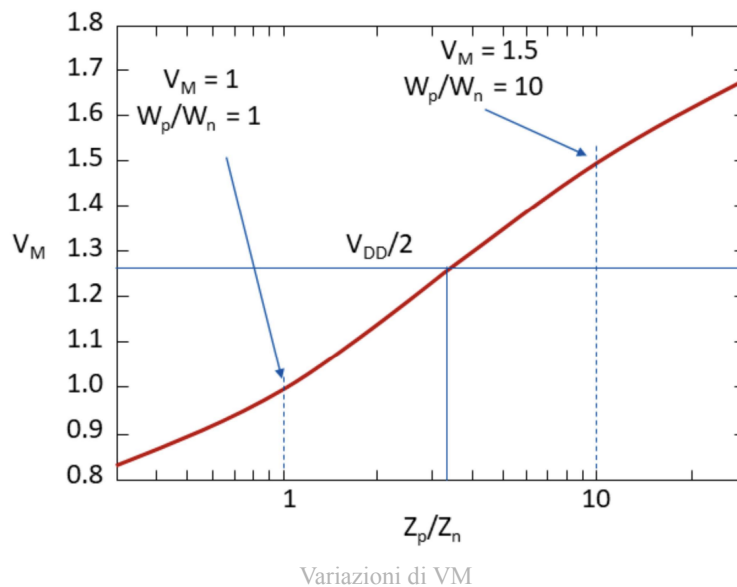
Spesso si vuole una caratteristica simmetrica, quindi $V_M = V_{DD}/2$. Inoltre piccole imprecisioni su r non hanno grandi ripercussioni.

Si può trovare la pendenza massima g della VTC per creare una versione lineare a tratti della VTC.

$$g = \frac{1 + r}{(\lambda_n - \lambda_p)(V_M - V_{TN} - \frac{V_{DSATN}}{2})}$$



La soglia di commutazione può variare leggermente in base ai dimensionamenti dei MOS. Infatti r dipende da Z_p/Z_n . Tipicamente il rapporto sta attorno a $3 \div 3.5$.

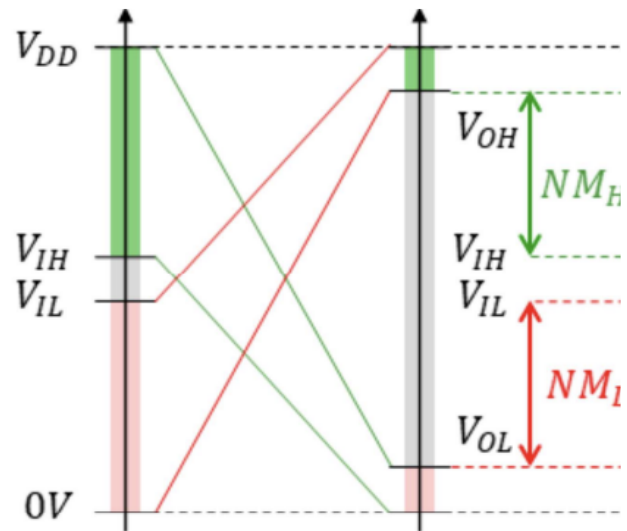


Nel grafico è chiaro come in condizioni di dimensioni uguali il NMOS è più forte del PMOS. Un aumento del NMOS porta ad abbassare V_M mentre un aumento del PMOS la alza.

Tolleranza Al Rumore

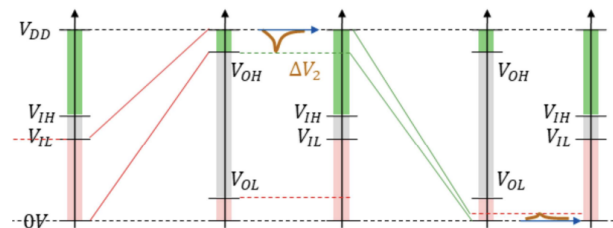
Come anticipato prima, la soglia logica non è definita da un valore specifico, bensì da un range. Questo è per contrastare al rumore che sballa il segnale. Una porta logica deve poter **rimuovere il rumore e rigenerare il segnale**.

Si possono definire i margini di rumore NM alti (H) e bassi (L) come l'ampiezza massima del disturbo tollerabile ad una porta. Per misurarli basta metterli in serie come in immagine



Noise Margin

La proprietà rigenerativa corregge il segnale purché il noise sia inferiore di NH . L'effetto è evidente in figura. V_M rappresenta i limiti del disturbo che possono essere attenuati.



Proprietà Rigenerativa

I margini sono:

$$NM_L = V_M$$

$$NM_H = V_{DD} - V_M$$

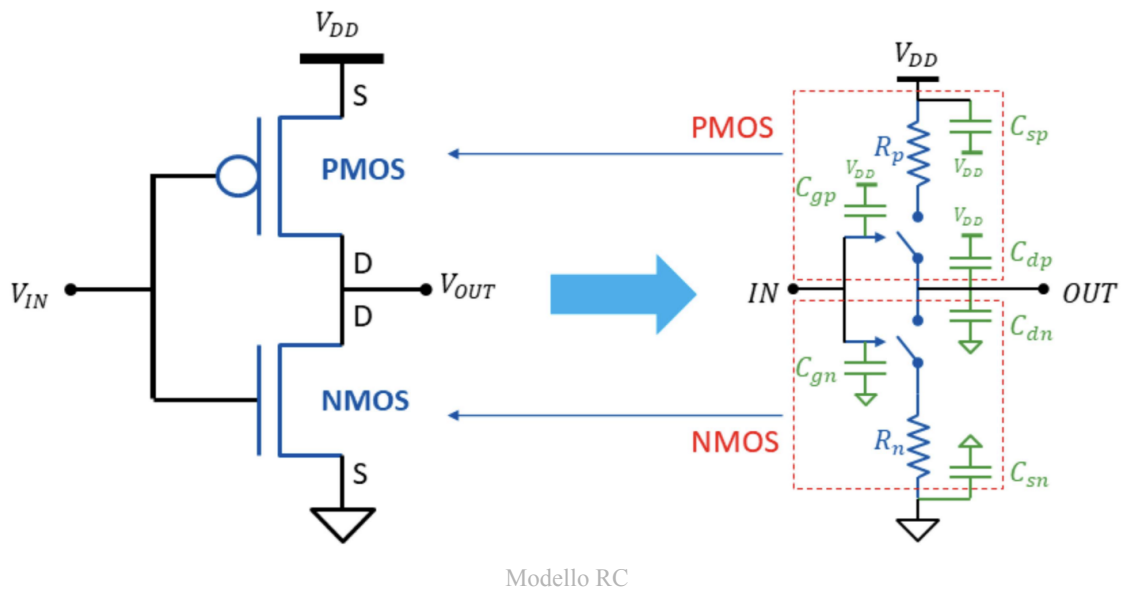
Remark.

Il dimensionamento ottimo in ottica di riduzione del rumore è che il **PMOS** sia **3 volte più largo**

$$\frac{Z_p}{Z_n} \approx 3$$

Modello RC

Questo modello tratta l'invertitore con le sue capacità parassite.



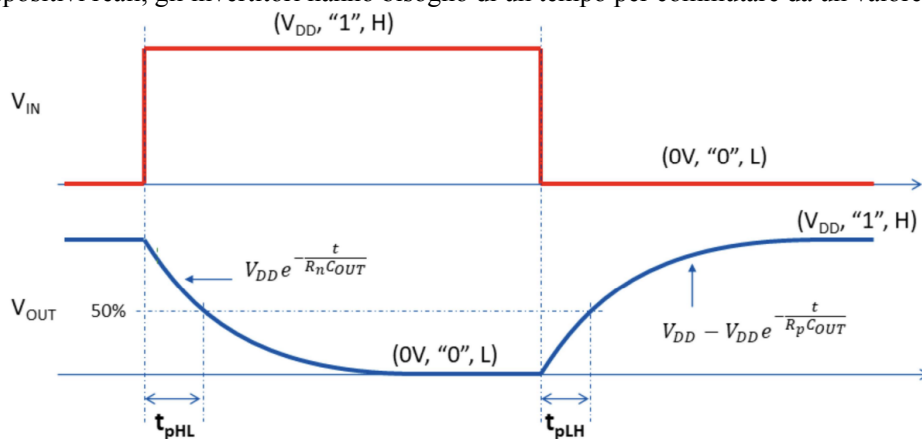
Qui possiamo definire i parametri d'ingresso e uscita:

Ingresso: $R_{in} = \infty$
 $C_{in} = C_{gn} + C_{gp}$

Uscita: $R_{out} = \frac{R_n + R_p}{2}$
 $C_{out} = C_{gn} + C_{gp}$

Tempi di Ritardo e Commutazione

Essendo dei dispositivi reali, gli invertitori hanno bisogno di un tempo per commutare da un valore all'altro.



Rappresentazione Tempo

Usando la formula di carica/scarica di un condensatore è possibile arrivare alle seguenti formule:

$$\begin{aligned} t_{pHL0} &= \ln(2) R_n C_{out} = 0.69 R_n C_{out} \\ t_{pLH0} &= \ln(2) R_p C_{out} = 0.69 R_p C_{out} \\ t_{p0} &= \frac{t_{pLH} + t_{pHL}}{2} = 0.69 \frac{R_n + R_p}{2} C_{out} \end{aligned}$$

Nel caso di una cascata i tempi di ritardo medi si andranno a sommare.

Data la richiesta di ottimizzare il tempo di propagazione si impostano le seguenti condizioni:

$$\begin{cases} \rho = \frac{R_{p0}}{R_{n0}} \\ \beta = \frac{Z_p}{Z_n} \end{cases} \xrightarrow{\text{opt}} \beta = \sqrt{\rho} \approx 1.4 \div 1.6 \implies Z_p = \sqrt{\rho} \cdot Z_n$$

Fan Out

Gli invertitori verranno infine implementati in dei circuiti, quindi al load avranno delle capacità aggiuntive, quindi andiamo a definire un valore chiamato **fan out** che va ad indicare quante porte "aggiuntive" stiamo pilotando

$$f = \frac{C_L}{C_{IN}}$$

nei tempi di ritardo utilizzeremo $C_{out} + C_L$ così da avere $t_{pLH0} + 0.69R_pC_L$ e $t_p = t_{p0} + 0.69R_{out}C_L$

Definendo un nuovo parametro:

$$\gamma = \frac{C_{out}}{C_{in}} = \frac{C_{d0}}{C_{g0}}$$

possiamo trovare una definizione del tempo di ritardo medio:

$$t_p = t_{p0} \left(1 + \frac{f}{\gamma} \right) = 0.69R_{out}C_{out} \left(1 + \frac{C_L}{C_{out}} \right)$$

Consumo Statico e Dinamico

L'energia assorbita per compiere un ciclo (in realtà si assorbe in LH e si dissipa in HL) è dato da

$$E = C_{out(tot)} \cdot V_{DD}(V_H - V_L)$$

Quella immagazzinata (in LH) è

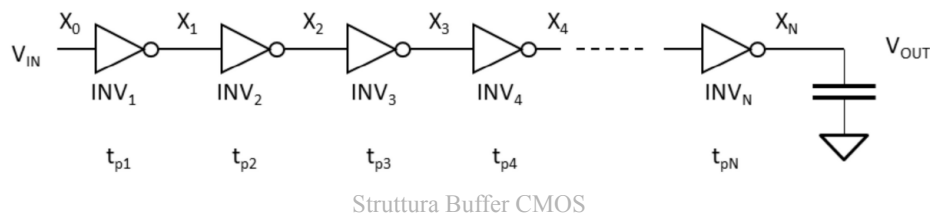
$$E = \frac{1}{2}C(V_H^2 - V_L^2)$$

Le energie si possono sommare, inoltre, data una frequenza fissa f , la potenza dinamica è data da

$$P_{dyn} = C_{out(tot)}V_{DD}(V_H - V_L)f$$

Buffer CMOS

Un buffer CMOS è una serie di n invertitori. Può essere invertente con n dispari e non-invertente con n pari.



Il tempo di propagazione complessivo sarà la somma di tutti i tempi di propagazione alternando tra LH e HL. Per ottimizzare il tempo t_{p0} intrinseco tutti gli invertitori avranno lo stesso $\beta = Z_p/Z_n$. Tuttavia vogliamo aumentare le capacità più avanziamo nella catena.

Come Ottenere Fan Out Ottimale

Per prima cosa trovo, se possibile, il dimensionamento ottimale del primo invertitore $Z_p = \sqrt{\rho}Z_n$

Trovo $C_{in(1)}$, C_{out} , t_{p0} senza considerare C_L

Lunghezza da scegliere

Step 1 ricorsivo: Impongo $\gamma = \frac{C_{out}}{C_{in}}$ e $f = 3, 6$.
Ripetere questa operazione fino a convergenza
 $f := \exp\left(1 + \frac{\gamma}{f}\right)$

Step 2: trovo fan out complessivo $F = \frac{C_L}{C_{in(1)}}$

Lunghezza predefinita

Step 1: trovo fan out complessivo $F = \frac{C_L}{C_{in(1)}}$

Step 2: trovo fan out $f = \sqrt[n]{F}$

Step 3: Lunghezza della catena di $N = \frac{\ln F}{\ln f}$

invertitori

Note bene: N determina se la catena è invertente o meno, se serve un tipo specifico si prendono i due valori più vicini e si calcola il tempo

$$t_{buff} = N t_{p0} \left(1 + \frac{f}{\gamma}\right) \text{ ottimale}$$

Step 4: trovo fan out $f = \sqrt[N]{F}$

infine sappiamo che l'invertitore i -esimo avrà dimensioni $Z_i = f Z_{i-1} = f^i Z_1$

Da qui il tempo di propagazione per ogni invertitore sarà

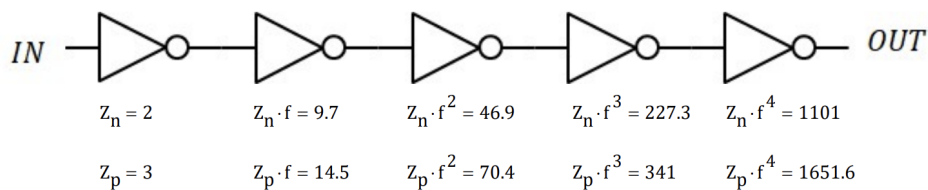
$$t_p = t_{p0} \left(1 + \frac{f}{\gamma}\right)$$

e quello totale

$$t_{buff} = N t_p$$

Il consumo dinamico è un'estensione di quello precedentemente descritto:

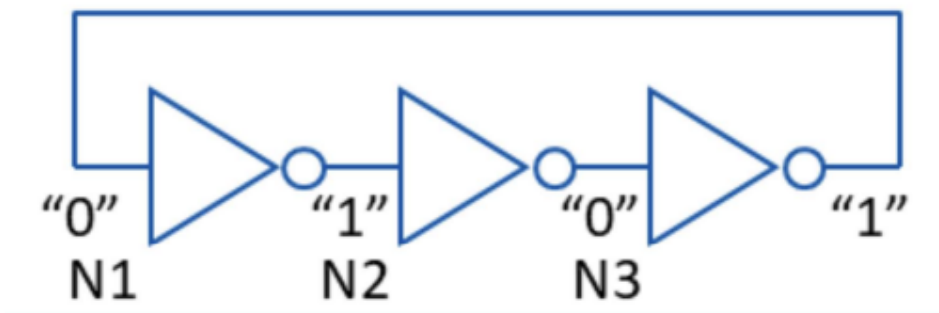
$$P_{dyn} = (C_{out} + f C_{in}) \left(\sum_0^{N-1} f^n \right) V_{DD}^2 f_0 = C_{in} V_{DD}^2 (\gamma + f) \frac{F-1}{f-1} f_0$$



Esempio Numerico

Oscillatore ad Anello

Un oscillatore è un dispositivo instabile che commuta continuamente tra 2 stati.



Oscillatore ad Anello

Le condizioni di oscillazione sono:

- N dispari (per avere asimmetria)
- $N t_{inv} > t_{r,f}$

Remark 1 (Frequency).

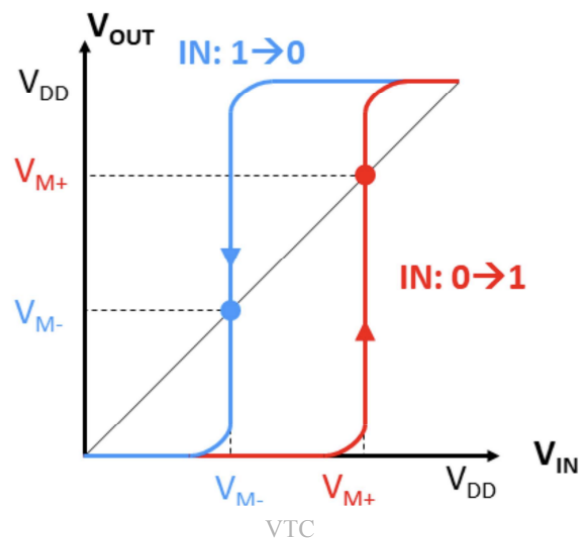
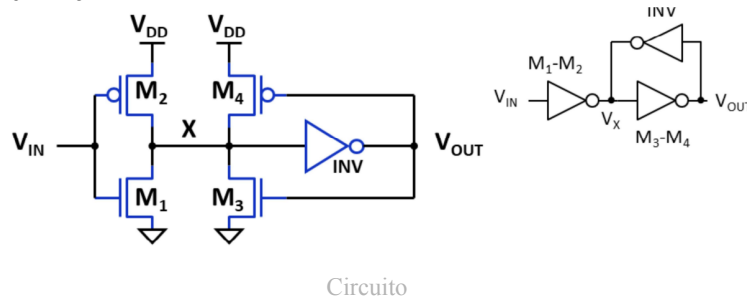
The frequency is defined as

$$f = \frac{1}{2Nt_{p0}}$$

since we must consider an entire $0 \rightarrow 1 \rightarrow 0$ cycle

Trigger di Schmitt

Il trigger di Schmitt è un preciso tipo di concatenazione di invertitori. Questo permette di creare una VTC diversa se abbiamo commutazione LH o HL



Analizziamo il comportamento a regime:

- $V_i = 0 \implies X = 1 \implies V_o = 0$: solo M_2, M_4 ON in parallelo
- $V_i = 1 \implies X = 0 \implies V_o = 1$: solo M_1, M_3 ON in parallelo

Analisi transizione LH $0 \rightarrow 1$

Inizialmente abbiamo $X = 1 > V_m$. Per commutare l'uscita X deve diminuire ma ha due fattori che contribuiscono:

- I_{DS1} che si accende e collega a massa
- $I_{DS2} + I_{DS4}$ che collegano X a V_{DD}
 M_2 va per spegnersi mentre M_4 è direttamente pilotato da INV e quindi rimane acceso
 V_{M+} va definita con $V_o = 0$ e quindi M_4 acceso, la soglia sarà più alta

****Analisi transizione HL $1 \rightarrow 0$**

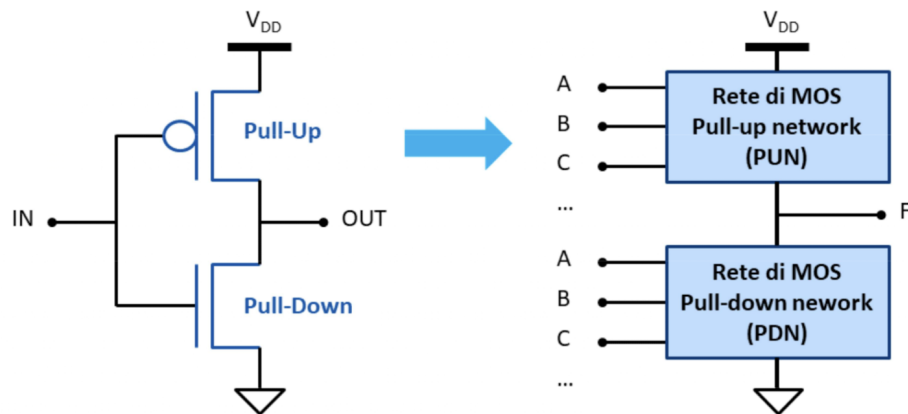
V_{M^-} va definita con $V_o = V_{DD}$ e quindi M_3 acceso, la soglia sarà più alta

I noise margin saranno

$$\begin{aligned} V_{NM_{HL}} &= V_{DD} - V_{M^-} \\ V_{NM_{LH}} &= V_{M^+} - 0V \end{aligned}$$

3) Logica Statica Complementare

Per logica **statica** si intende una logica con sempre un voltaggio di riferimento, per complementare si intende di avere due reti di n/p MOS separate dove solo una delle due è accesa. In questo capitolo vedremo una specie di "invertitore potenziato" dove ogni entrata andrà nelle due reti dell'invertitore



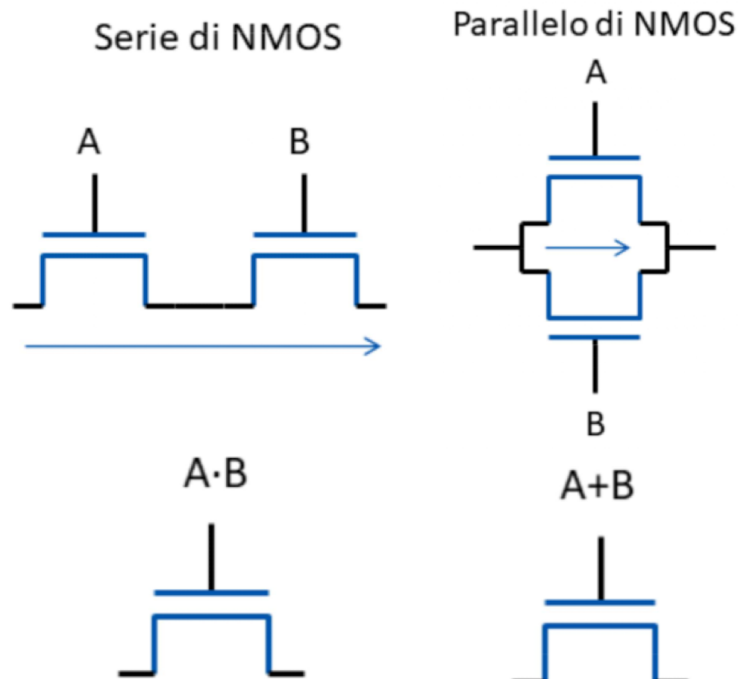
Rappresentazione

Per prima cosa bisogna effettuare un remark:

Remark.

Gli NMOS trasmettono bene valori bassi e male quelli alti. Inoltre se usati come interruttore da spenti $V_G = 0$ sono un circuito aperto. Per gli PMOS è l'opposto, trasmettono bene i valori alti, male quelli bassi e sono accesi finché non viene applicata una corrente al gate per farli spegnere. Infine possiamo affermare che a livello logico **i PMOS funzionano a logica *negata* mentre i NMOS a logica normale**

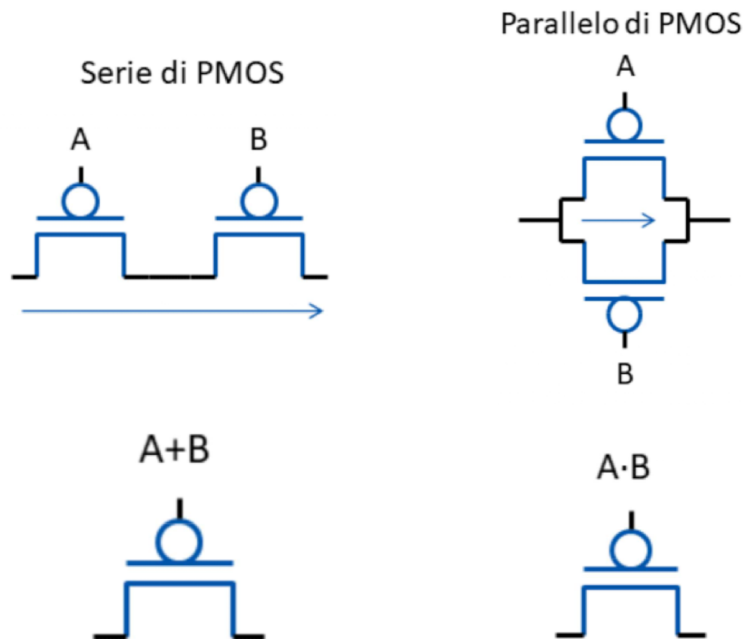
Serie e Paralleli di NMOS



Schema

Il ragionamento è molto semplice. Nella serie entrambi (AND) devono essere accesi per far passare il segnale, nel parallelo basta che ce ne sia almeno uno (OR).

Serie e Paralleli di PMOS



Schema

In questo caso abbiamo il comportamento opposto. La serie funziona finché almeno uno dei due non si spegne (OR), mentre il parallelo funziona finché entrambi non si spengono (AND).

Con queste informazioni è possibile costruire ogni tipo di rete.

Reti Duali

Definition 2 (reti Duali).

Date due reti controllate dai medesimi N segnali, le reti sono dette **duali** se per ognuna delle 2^N combinazioni una e solo una delle reti è accesa.

Il duale di una serie di NMOS è un parallelo di PMOS e per un parallelo di NMOS c'è la serie di PMOS, da qui è possibile costruire ogni rete duale partendo da una rete qualsiasi.

Remark.

Sempre in ambito di NMOS e PMOS, sapendo un'espressione di una rete NMOS basta negarla e utilizzando De Morgan si può arrivare all'espressione della rete duale PMOS. Per esempio

$$A \cdot (B + C) \xrightarrow{\text{PMOS duale}} \overline{A \cdot (B + C)} = \overline{A} + \overline{B + C} = \overline{A} + \overline{B} \cdot \overline{C}$$

Nell'invertitore PDN e PUN devono essere duali. La PDN è collegata a massa, la PUN a V_{DD} . PUN avrà come output F , mentre PDN avrà come output $\overline{F}$

Porte Logiche Elementari

L'approccio è quello di costruire la truth table con input, output F e analisi degli stati di PDN e PUN. Poi si costruisce la PDN e si trova la PUN

NAND

$$F = \overline{A \cdot B}$$

A	B	F	PDN
0	0	1	OFF
0	1	1	OFF
1	0	1	OFF
1	1	0	ON

Da qui PDN sarà una serie di due NMOS e PUN un parallelo di due PMOS

NOR

$$F = \overline{A + B}$$

A	B	F	PDN
0	0	1	OFF
0	1	0	ON
1	0	0	ON
1	1	0	ON

Qui è più semplice notare che PUN è un AND e quindi una serie di due PMOS, al che il PDN è un parallelo di due NMOS.

Example.

Una porte più complessa

$$F = \overline{A \cdot B + C \cdot (D + E)}$$

Studio $\overline{F} = A \cdot B + C \cdot (D + E)$ e capisco come costruire gli input: $A \cdot B$ in parallelo con $C \cdot (D + E)$ che diventa (A in serie con B) questo in parallelo con [C in serie con (D in parallelo con E)]. Da qui si trova facilmente l'espressione del PUN.

Remark.

Con questo approccio è possibile costruire solo porte logiche invertenti, per costruirne di non invertenti ci sono 2 approcci

- Coniugare le entrate con N invertitori

- Coniugare l'uscita con 1 invertitore

Funzioni Invertenti e Non Invertenti

✓ Todo

Caratteristiche Statiche

La tensione di soglia logica V_M può essere definita diversamente in base a come vengono fissati gli input. Dato che la variazione di TSL porta a cambiamenti poco piccoli si può semplificare il calcolo supponendo tutti gli input attivi, trovando il Z_{eq} e usando il calcolo "tradizionale".

Caratteristiche Dinamiche

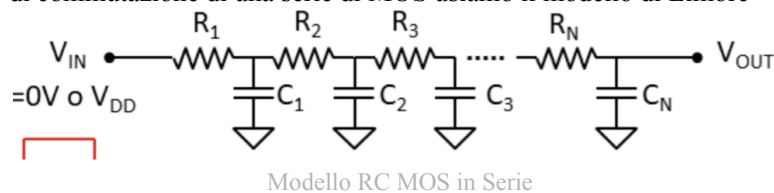
Le capacità parassite sono calcolate analogamente al caso dell'invertitore, ma ora ci saranno più porte connesse all'uscita. Idem per le capacità parassite in entrata, ma qui si andranno a definire capacità parassite per ogni entrata. Vanno però definite capacità parassite nei nodi intermedi (che possono avere effetto body!).

Le commutazioni vanno considerate sempre nel caso peggiore (percorso più lungo e con meno numero di MOS in parallelo)

Nel caso di una serie di MOS ci saranno le capacità dei nodi intermedi che andranno a sottrarre corrente di scarica da C_F

MODELLO DI ELMORE

Per calcolare il tempo di commutazione di una serie di MOS usiamo il modello di Elmore



$$t_p = 0.69 \sum_{i=1}^N \left(C_i \sum_{j=1}^i R_j \right)$$

Con R_n e R_p costanti è importante notare che una serie di N corrisponderà ad avere un parallelo di N nella rete opposta, quindi possiamo fare le seguenti supposizioni:

- Ultima capacità della serie: $C_N = C_{out}$
 - Capacità di uscita: $C_{out} = C_{dn} + NC_{dp}$
 - Commutazione LH: $t_{pLH} = 0.69(C_{out} + C_L)$
 - Commutazione HL: $t_{pHL} = 0.69R_n(C_1 + 2C_2 + \dots + NC_{out}) + 0.69NR_nC_L$
- da qui si trova t_p

$$t_p = 0.69 \frac{R_n(C_1 + \dots + NC_{out}) + R_pC_{out}}{2} + 0.69 \frac{NR_n + R_p}{2} C_L = t_{p0} + 0.69R_{out}C_L$$

Notare come $NC_{out} = O(N^2)$ e quindi il tempo di commutazione è parabolico!

OTTIMIZZAZIONE

Dimensionamento progressivo

Questo consiste nel avere Z_N più piccolo di tutti e man mano aumentare la grandezza di un fattore α

Ordine degli ingressi

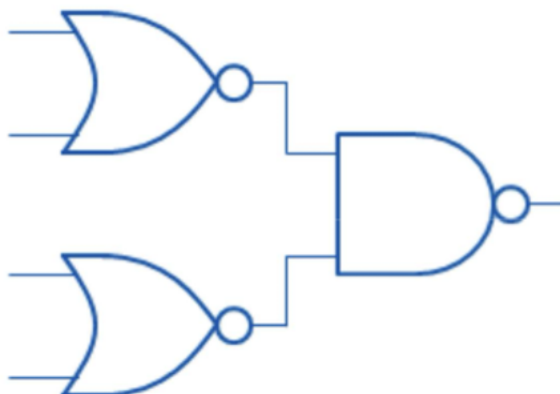
Questo consiste in selezionare in che ordine nella serie sono connessi gli input in base a parametri di costruzione tipo tempo di ritardo o probabilità di accensione.

Bilanciamento PUN PDN

Il bilanciamento consiste nel avere le resistenze equivalenti dei due networks uguali. Si guarda sempre il caso peggiore. Una volta trovato il rapporto tra Z_p e Z_n si può trovare sapendo il rapporto $R_{p0}/R_{n0}(\approx 2)$

Inoltre se abbiamo troppi ingressi conviene ridurre il numero di porte per stadio usando una tecnica di doppia negazione e De Morgan. Esempio

$$A + B + C + D = \overline{\overline{A + B + C + D}} = \overline{(A + B) \cdot (C + D)}$$



Or a 4 ingressi con porte da 2 ingressi

I tempi di commutazione si possono sommare algebricamente

Consumo

Ricordando la definizione di consumo dinamico aggiungiamo un termine

$$P_{dyn} = C_F V_{DD} (V_H - V_L) f \alpha_f$$

Questo va ad indicare il **fattore di attività**, ovvero la probabilità che ci sia una commutazione dell'uscita. Ad esempio se $p_A = 0$ $A \cdot B = 0$ e quindi la porta non commuterà mai

Funzione logica	P_F	Fattore di attività
AND	$P_A P_B$	$P_A P_B \cdot (1 - P_A P_B)$
NAND	$1 - P_A P_B$	$P_A P_B \cdot (1 - P_A P_B)$
OR	$p_A + (1 - p_A)p_B$	$[p_A + (1 - p_A)p_B] \cdot [(1 - p_A)(1 - p_B)]$
NOR	$(1 - p_A)(1 - p_B)$	$[p_A + (1 - p_A)p_B] \cdot [(1 - p_A)(1 - p_B)]$
XOR	$(p_A + p_B - 2p_A p_B)$	$(p_A + p_B - 2p_A p_B) \cdot (1 - p_A - p_B + 2p_A p_B)$
XNOR	$1 - p_A - p_B + 2p_A p_B$	$(p_A + p_B - 2p_A p_B) \cdot (1 - p_A - p_B + 2p_A p_B)$

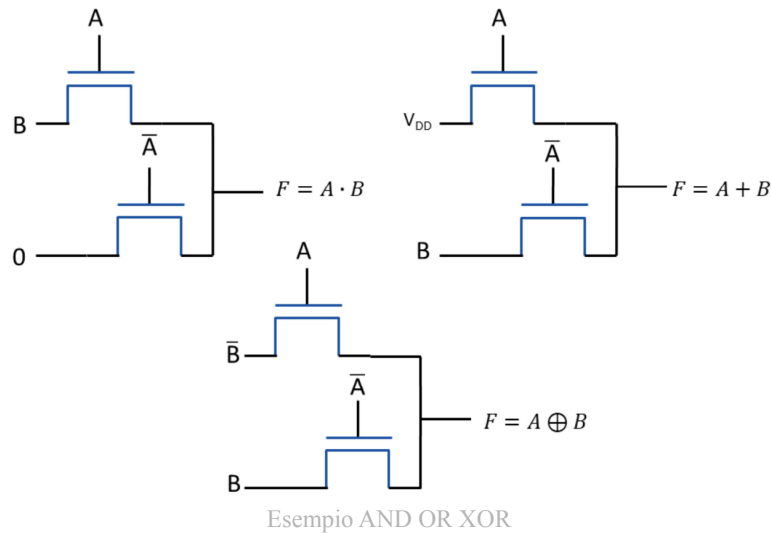
Fattori di Attività Notevoli

Dove $\alpha_f = P_F(1 - P_F)$.

4) Porte Logiche Pass Transistor

Una porta pass transistor è una connessione di MOS dello stesso tipo che ha funzione da MUX.

Questo approccio permette l'utilizzo di meno MOS rimanendo comunque complementare in quanto i MOS sono pilotati al gate da un input e dal suo negato



In generale chiamiamo il segnale al gate S e gli altri due A,B. Con dei NMOS avremo $F = SA + \bar{S}B$

Remark.

Alcune identità da sapere:

$$\begin{aligned} A\bar{B} + \bar{A}B &= A \oplus B & AB + \bar{A}\bar{B} &= \overline{A \oplus B} \\ A + BC &= (A + B)(A + C) & A + \bar{A}B &= (A + \bar{A})(A + B) = A + B \\ \overline{A + B} &= \bar{A} \cdot \bar{B} & A + B &= \overline{\bar{A} \cdot \bar{B}} \end{aligned}$$

Ora possiamo trovare un modo per costruire porte Pass-T arbitrarie:

Theorem 3 (Teorema di Shannon).

Ogni funzione logica a N variabili $F(A, B, C, \dots)$ può essere riscritta come

$$\begin{aligned} F(A, B, C, \dots) &= AF_1(B, C, \dots) + \bar{A}F_0(B, C, \dots) \\ &= AF(1, B, C, \dots) + \bar{A}F(0, B, C, \dots) \end{aligned}$$

Questo teorema è alla base della logica Pass-T e può essere applicato ricorsivamente.

Implementazione a pass-T

$$F(A, B, C) = \bar{B} + A \cdot C$$

$$F(A, B, C) = \bar{A} \cdot F_0(B, C) + A \cdot F_1(B, C)$$

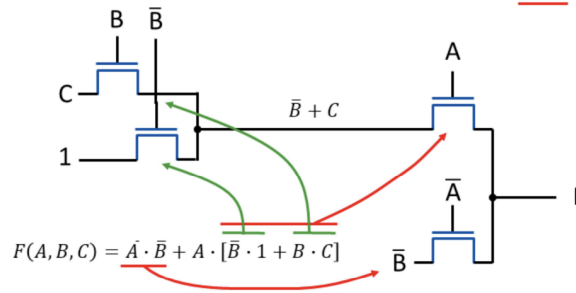
$$F_0(B, C) = \bar{B} + 0 \cdot C = \bar{B}$$

$$F_1(B, C) = \bar{B} + 1 \cdot C = \bar{B} + C$$

$$F_1(B, C) = \bar{B} \cdot F_{10}(C) + B \cdot F_{11}(C)$$

$$F_{10}(C) = 1$$

$$F_{11}(C) = C$$



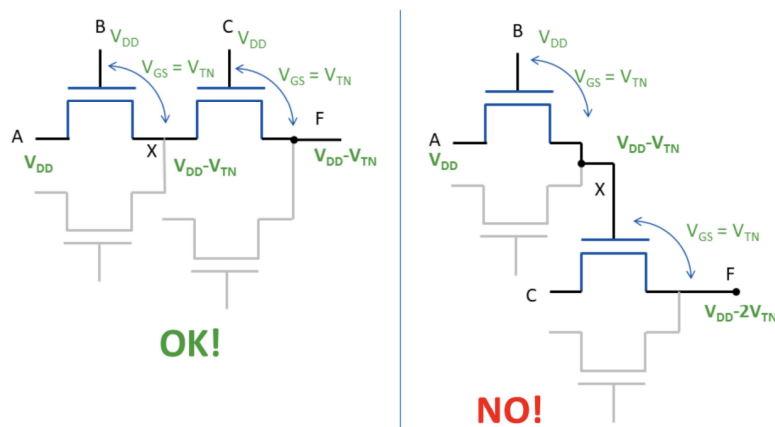
Esempio

Uno studio delle caratteristiche statiche e dinamiche mostra molti svantaggi in questa applicazione (no error correction, poca noise reduction, trasmissione peggiore di valori alti o bassi). In seguito vedremo come aggirare questi problemi

Caratteristiche Statiche

Ovviamente per una porta realizzata da 2 MOS il valore di uscita alto o basso sarà trasmesso male se usiamo NMOS o PMOS rispettivamente.

Quindi è importante che **gli output dei vari stadi non siano usati come input dei gate agli stadi successivi** e quindi una connessione in serie è l'unica implementabile



Esempio

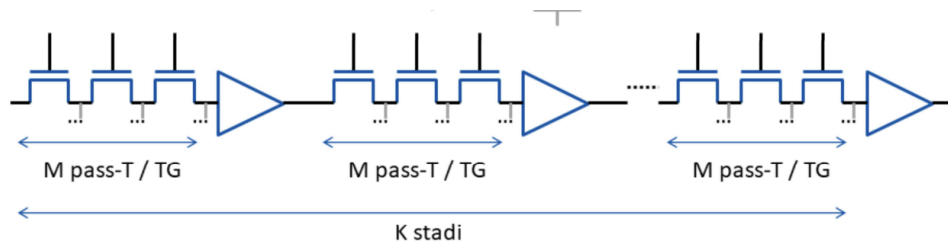
Se usiamo l'uscita per pilotare un invertitore l'entrata del segnale non sarà V_{DD} e quindi i due MOS rimarranno accesi, l'invertitore avrà un consumo statico non nullo! Fortunatamente grazie alle proprietà dell'invertitore il valore alto degradato verrà corretto (proprietà rigenerativa)

Caratteristiche Dinamiche

Una connessione in serie di invertitori consente di sommare i tempi di propagazione poichè le correnti non passeranno agli altri stadi data l'impedenza infinita del gate. Questo però non si presenterà nei Pass-T in quanto tra i vari stadi c'è una connessione diretta e si dovrà usare il modello di Elmore (non sempre!).

Ottimizzazione Dei Valori Logici (Caratteristiche Statiche)

Per ottimizzare la rete di Elmore è necessario aggiungere un buffer ripetitore lungo la catena



Pasted image 20250402134527.png

Il numero ottimale di buffer può essere calcolato usando la seguente formula (anche se C dell'ultima capacità è diversa dai nodi interni)

$$M = 1.7 \sqrt{\frac{t_{buf}}{RC}}$$

con C la capacità di un nodo ed R la resistenza di MOS. Infine il tempo di propagazione sarà:

$$t_p = k \left[0.69 \cdot RC \cdot \frac{M(M+1)}{2} + t_{buf} \right] = 0.69 \cdot RC \cdot \frac{N(M+1)}{2} + \frac{N}{M} t_{buf}$$

5) Layout

Lmao non lo faccio, mi serve solo per avere l'indicizzazione dei capitoli corretta

6) Blocchi Logico Aritmetici

In questo capitolo discuteremo alcuni circuiti specifici:

Decoder Binario

Il decoder è un circuito a N ingressi e 2^N uscite. L'uscita che ci accenderà corrisponde al valore decimale dell'input in binario. Con una tabella di verità è possibile trovare l'implementazione, sia in logica diretta (NOR) che negata (NAND). Si può dimostrare che

$$t_p \sim O(N^2)$$

Per migliorare il tempo di propagazione si può suddividere un decoder in più stadi alternando tra implementazioni a NOR e NAND. Questo si può fare perchè ogni funzione logica può essere raggruppata come pezzetti di altre funzioni logiche usando de Morgan:

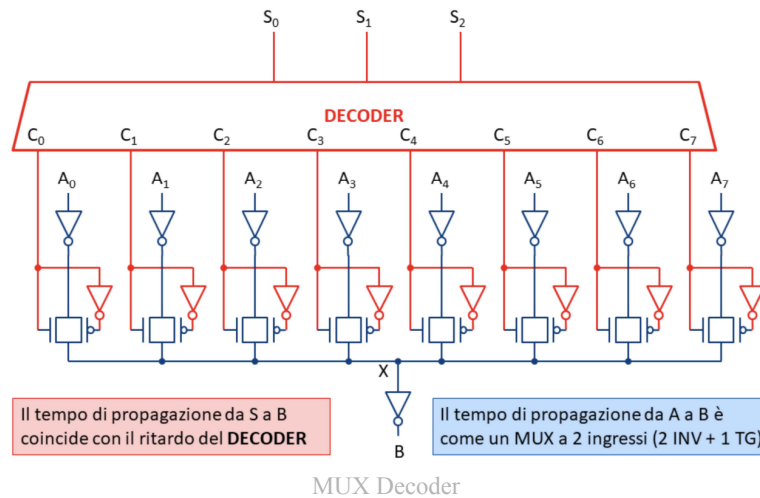
$$A \cdot B \cdot C \cdot D = (A \cdot B) \cdot (C \cdot D) = \overline{\overline{(A \cdot B)} + \overline{(C \cdot D)}}$$

MUX

Ricordando la struttura del MUX 2:1 è possibile costruire dei MUX $N:1$. Basta scrivere la funzione logica, e raggruppare in forma $F = S_0 \cdot X + \bar{S}_0 \cdot X$ e così via (**struttura ad albero**).

In questa implementazione serviranno $M = \log_2 N$ stadi. Inoltre gli invertitori intermedi possono essere eliminati in quanto un MUX 2:1 ne ha uno sia in entrata che in uscita.

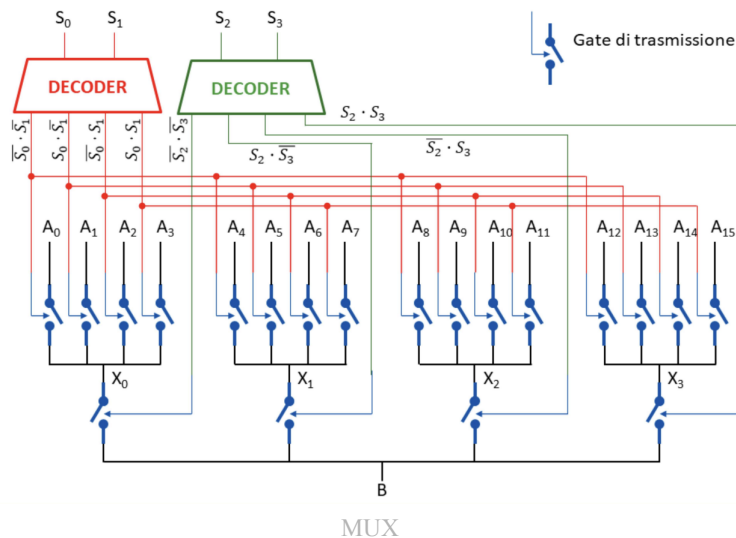
Alternativamente è possibile implementare un MUX tramite un **decoder**:



Il tempo di propagazione al nodo X dipende dalle alte capacità della serie di N pass tg in uscita ai segnali:

$$t_p = t_{p,mux} + 0.69R \cdot NC$$

Infine si possono mischiare le due implementazioni:



Dividendo in K gruppi da $M=N/K$ bit. Ogni gruppo avrà 2^M uscite. La struttura ad albero avrà K TG in serie. Si può dimostrare che

$$t_p = 0.69R(2^M + 1)CK(K + 1) \approx 0.69 \cdot 2^M \frac{N^2}{M^2}$$

ed è minimizzato con $M = 2/\ln 2 \approx 3$.

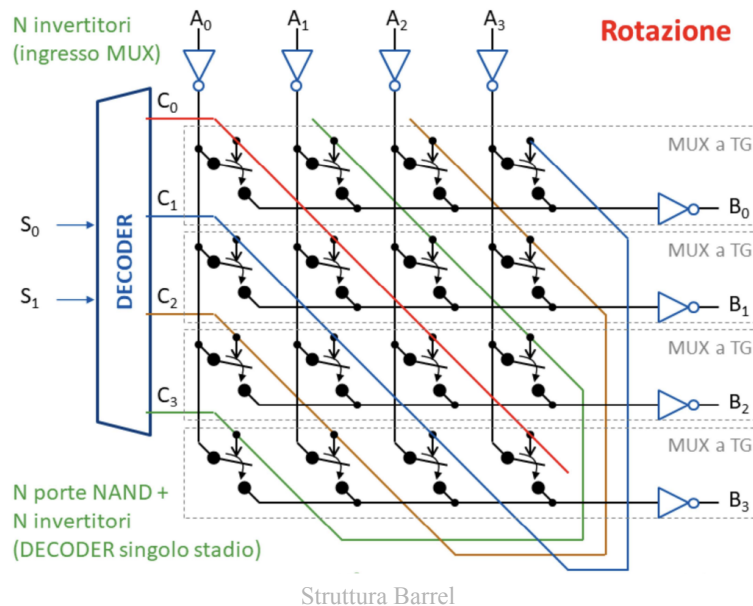
Traslazione/Rotazione

Una traslazione a SX di M posizioni equivale a

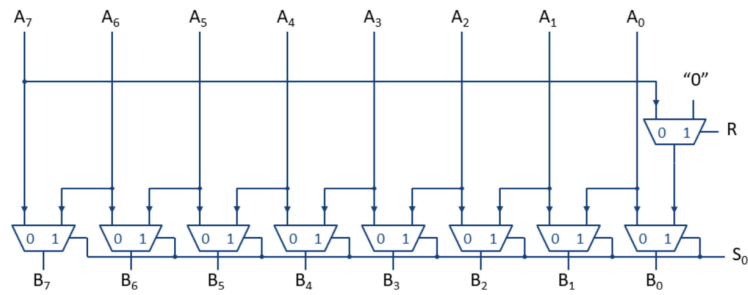
- Moltiplicare per 2 senza estensione del segno
- Dividere per 2 con estensione del segno

Una rotazione di M bit a SX equivale ad una rotazione di N-M bit a dx.

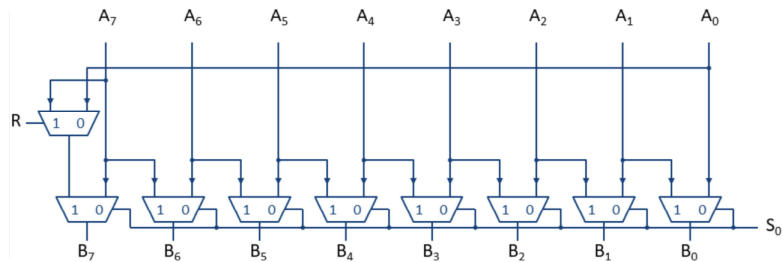
Un Traslatore è realizzabile con **struttura a barrel** tramite un decoder che i TG delle entrate alle uscite corrette.



Invece è possibile anche un implementazione a **struttura logaritmica** In cui si eseguono più rotazioni separate ma con M piccoli. Gli step sono decisi dalla codifica binaria del numero (Es: $5_{10} = 101_2 \rightarrow 1$ rotazione di 4 bit, 0 di 2 bit, 1 di 1 bit)



Traslazione/Rotazione 1 o 0 bit a SX



Traslazione/Rotazione di 1 o 0 bit a DX

Se $R=1$ l'operazione svolta è una rotazione. Per generalizzarlo a N-bit di rotazione L'entrata A_x sarà collegata al MUX spostato x a sinistra/destra. Chiaramente nel caso di una rotazione bisognerà collegare le uscite limitrofe ai MUX pilotati da R opportunamente

6.2) Sommatore

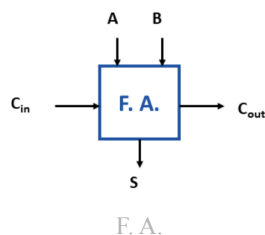
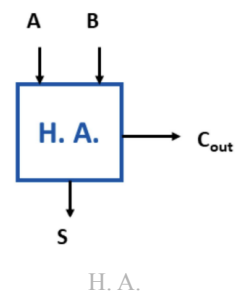
Il sommatore è così importante da meritarsi un subchapter dedicato. Infatti il sommatore è alla base della somma (duh), **sottrazione** (dato che $A - B = A + \overline{B} + 1$), moltiplicazione, comparazione.

Recap di HA, FA

Un Half Adder somma due bit A,B e ha come output il risultato S e il riporto C

$$F = A \oplus B$$

$$C = A \cdot B$$



Il Full Adder invece è un HA che ce l'ha fatta e tiene conto anche di un riporto esterno

$$F = A \oplus B \oplus C_{in}$$

$$C = A \cdot B + (A \oplus B)C_{in}$$

$$= A \cdot (\overline{A \oplus B}) + C_{in} \cdot (A \oplus B)$$

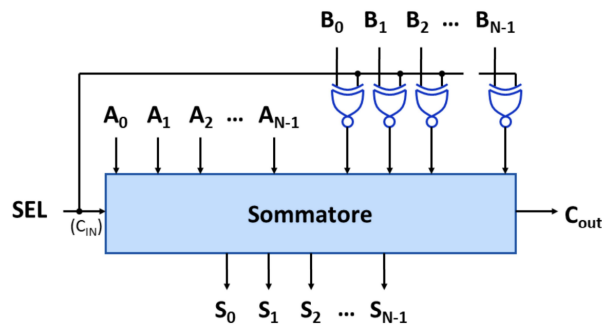
Sottrattore

Come scritto in precedenza per svolgere un sottrattore basta ricordarsi la seguente proprietà

Theorem 4 (Sottrazione = Somma).

Con A, B due numeri binari vale la seguente proprietà

$$A - B = A + \overline{B} + 1$$



Sommatore E Sottrattore

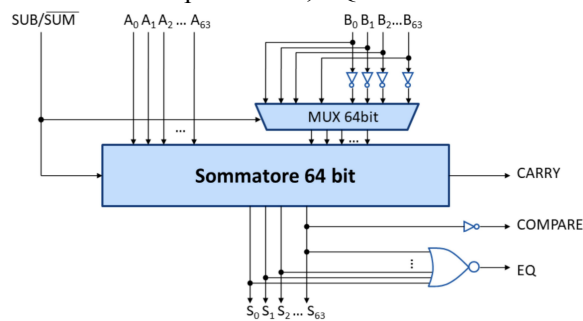
Comparatore

L'uguaglianza corrisponde ad una XNOR e può essere costruita tramite un implementazione ad albero. Utilizzando un sommatore invece è possibile svolgere anche disuguaglianze

- Sommatore Compare: L'ultimo bit (MSB) sarà:

$$\text{COMPARE} = \begin{cases} 1 & \text{if } A \geq B \\ 0 & \text{if } A < B \end{cases}$$

- Sommatore Eq: Unendo tutte le uscite ad una porta NOR, EQ=1 se A=B

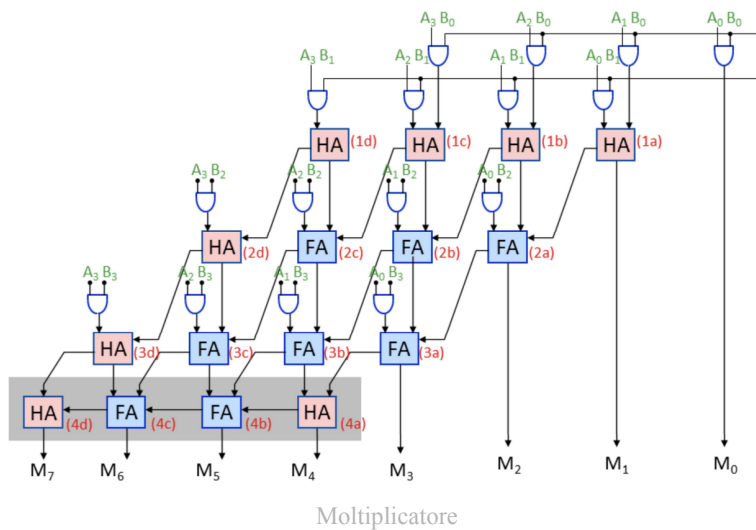


Blocco Sommatore/Sottrattore/Comparatore

Moltiplicatore

Per eseguire una moltiplicazione tra N bit bisogna svolgere N-1 somme

Prima bisogna fare il prodotto delle cifre di A con una cifra di B per ogni cifra di B, infine sommare questi prodotti parziali



un prodotto di numeri a N bit avrà bisogno di N stadi. Quindi si dovrà aspettare

- 1 and poichè iniziano tutti in parallelo
- N-1 riporti poichè ogni stadio finisce/inizia in parallelo e da il riporto per il prossimo stadio
- 1 somma di N bit dato che l'ultimo stadio richiede una somma "classica"

$$t_{mult} = t_{and} + (N - 1)t_c + t_{sum}$$

6.2.2) Ottimizzazione

Ora ci occuperemo di implementare sommatore il più possibilmente performanti.

Propagate e Generate

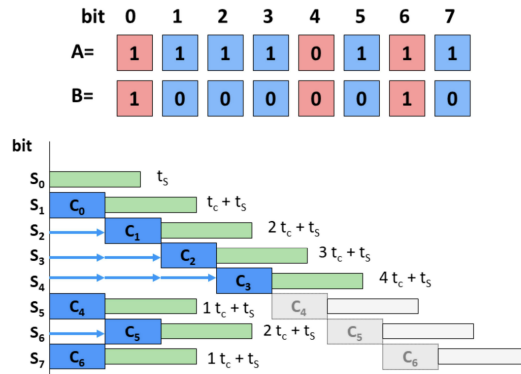
Guardando la tabella di verità di un FA si possono individuare 3 regioni distinte che in pratica hanno due comportamenti opposti, ovvero che **una zona non ha bisogno di sapere il risultato della somma precedente per calcolare C_{out}**

Tabella di verità						
A	B	C_{in}	S	C_{out}	Condizione	Operatore logico
0	0	0	0	0	DELETE, CLEAR o GENERATE-0	A=0, B=0 $\overline{A \cdot B}$
0	0	1	1	0		
0	1	0	1	0	PROPAGATE	A=1, B=0 A=0, B=1 $A \oplus B$
0	1	1	0	1		
1	0	0	1	0		
1	0	1	0	1		
1	1	0	0	1	SET, o GENERATE-1	A=1, B=1 $A \cdot B$
1	1	1	1	1		

Propagate Generate Truth Table

Questo significa che un sommatore può iniziare a sommare in parallelo da ogni GENERATE. Inoltre le zone sono

determinate da A e B, non da C_{in}



Esempio maggiore efficienza

Così è possibile costruire un F.A. in modo da calcolare anche i Generate o Propagate

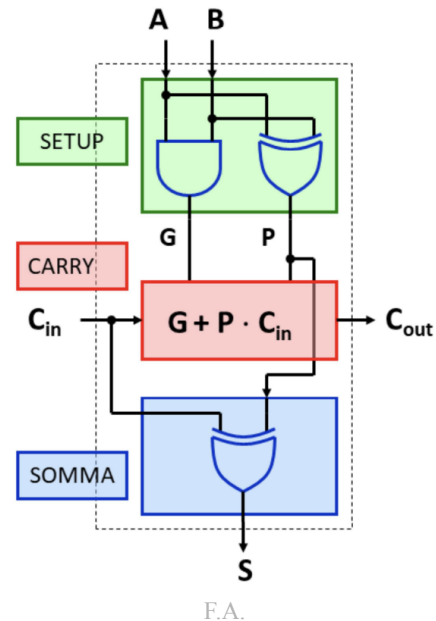
$$\text{Propagate: } P = A \oplus B$$

$$\text{Generate: } G = A \cdot B$$

$$S = P \oplus C_{in}$$

$$C_{out} = G + P \cdot C_{in}$$

Così è evidente che $C_{out} = 1$ se $G = 1$



Ripple Carry

Collegando N FA in serie (partendo da LSB) è possibile creare un sommatore di due numeri a N bit.

Un implementazione reale consiste nel creare il circuito $P = A \oplus B$ e poi $F = P \oplus C_{in}$ e $C = PC_{in} + \overline{P}A$. I due tempi che ci interessano sono quelli per ottenere la somma $T_S = T_{S_N}$ e quello per ottenere il riporto: $T_C = T_{C_N}$

In un ripple carry abbiamo che:

$$T_S = (N - 1)t_c + t_s$$

$$T_C = Nt_c$$

Sfruttando le proprietà del generate invece è possibile trovare una leggerezza ottimizzazione (non valida nel caso peggiore). Le catene di propagate vengono tutte calcolate in parallelo, definendo con M_i il numero di propagate che seguono il generate i-esimo possiamo trovare i seguenti tempi:

$$T_{c_i} = M_i t_c$$

$$T_{c_{\max}} = M_{\max(i)} \cdot t_c$$

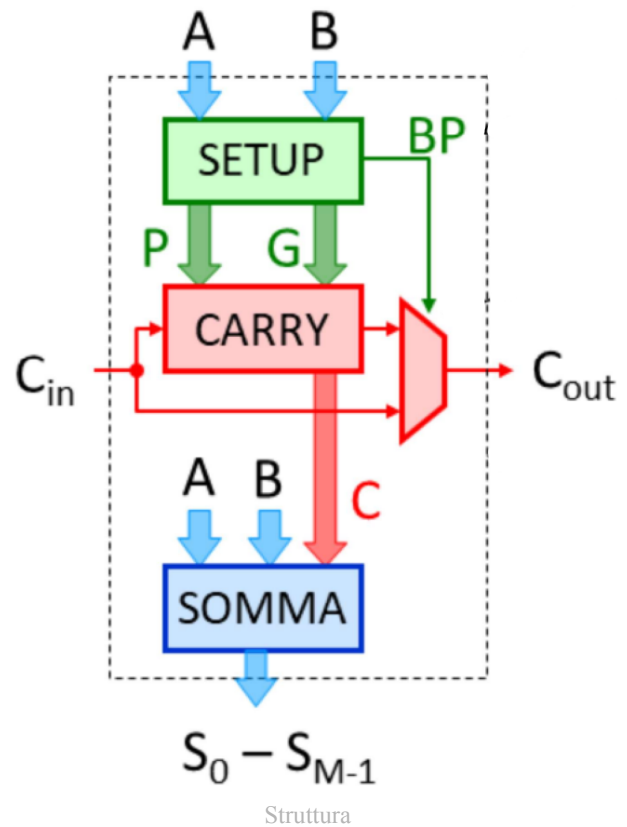
$$T_{s_i} = (M_i - 1)t_c + t_s$$

$$T_{c_{\max}} = (\max_i \{M_i\} - 1)t_c + t_s$$

Carry By-Pass

Da adesso inizieremo a considerare le proprietà dei propagate/generate. In quest'implementazione gli N bit vengono raggruppati in K blocchi da $M=N/K$ bit.

Il segnale BP è un NAND di tutti i P del blocco. Se il blocco è fatto di soli propagate allora C_{in} del blocco salterà tutti i calcoli e andrà direttamente al prossimo blocco.



- Tutti i blocchi iniziano in *parallelo* quindi t_{setup} sarà impiegato una volta.
- Ogni *generatore lavorerà in parallelo* per determinare l'uscita del MUX. Il tempo necessario al blocco generatore per il riporto finale è $M \cdot t_c$. Inoltre ogni blocco deve tenere in considerazione t_{MUX} la cui uscita dipenderà degli input quindi è in *serie*. (caso peggiore se il primo è generate e tutti i restanti propagate)

$$t_{carry} = t_{setup} + Mt_c + Kt_{mux}$$

- La somma dipende da tutti i riporti del blocco, che a loro volta dipendono da quelli del blocco prima. Per completare la somma bisogna che venga calcolato l'ultimo bit, che nel caso peggiore è in un blocco di propagate, quindi il tempo sarà dato dal tempo di quel carry+l'arrivo del carry all'ultimo bit e la somma. Infine l'ultimo MUX è inutile ai fini della somma.

$$\begin{aligned} t_{sum} &= t_{setup} + Mt_c + (K-1)t_{mux} + (M-1)t_c + t_s \\ &= t_{setup} + (2M-1)t_c + (K-1)t_{mux} + t_s \end{aligned}$$

La scelta ottimale di M e K si ottiene ottimizzando il tempo critico (t_{sum}) derivando rispetto a M.

$$M_{opt} = \sqrt{\frac{N \cdot t_{mux}}{2t_c}} \stackrel{t_{mux} \approx t_c}{=} \sqrt{\frac{N}{2}}$$

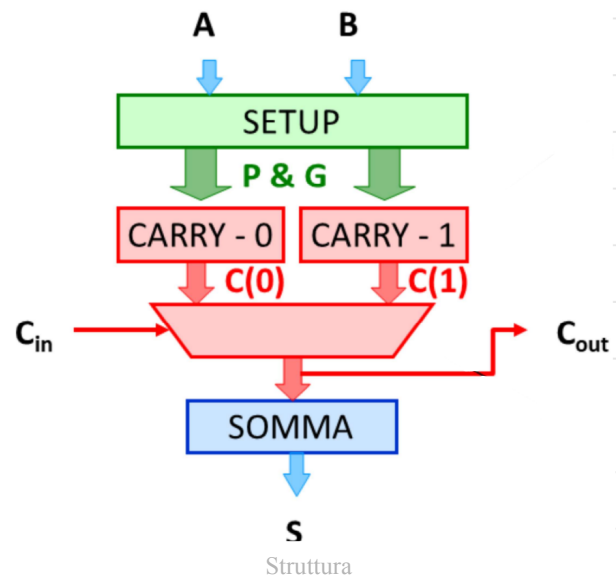
Remark.

Ricorda che nel blocco la somma verrà calcolata partendo da C_{in} (anche se il blocco viene bypassato, il riporto deve arrivare fino all'ultimo bit). Quindi il tempo di somma dovrà tenere conto di questo

Linear Carry Select

Il linear carry select è molto simile al CBP solo che invece di attendere C_{in} calcola entrambi i casi immediatamente

In questo caso, non solo t_{setup} ma anche $M \cdot t_c$ sono calcolati in parallelo fin dall'inizio.



- Il tempo per il riporto finale allora dipenderà solo da

$$t_{carry} = t_{setup} + Mt_c + Kt_{mux}$$

- Il tempo per la somma avrà in aggiunta anche il tempo per eseguire la somma:

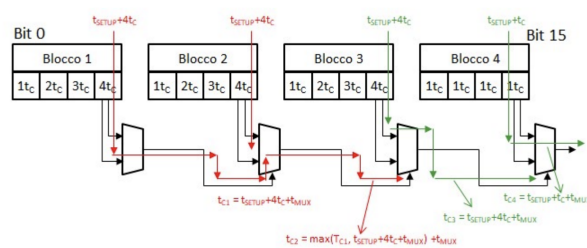
$$t_{sum} = t_{setup} + Mt_c + Kt_{mux} + t_s$$

L'ottimizzazione da il seguente risultato:

$$M_{opt} = \sqrt{\frac{N \cdot t_{mux}}{t_c}} \stackrel{t_{mux} \approx t_c}{=} \sqrt{N}$$

Il caso peggiore si ottiene se si ha la seguente sequenza: *GPPP...*

Exercise (Esercizio).



Esempio

Nella seguente implementazione abbiamo un LCS a 16 bit con 4 bit a blocco. La somma è di poco conto, ci interessa sapere che i bit danno i seguenti risultati:

GPPP PPPP GPPP GGGG

Per svolgere questi esercizi conviene scrivere il tempo di carry di ciascuno stadio, tenendo conto se esso ha o meno un G.

(G)	$t_{C1} = t_{setup} + 4t_C + t_{mux} = 4.7$	[ns]
(P)	$t_{C2} = \max[t_{C1}, t_{setup} + 4t_C] + t_{mux} = t_{setup} + 4t_C + 2t_{mux} = 4.9$	[ns]
(G)	$t_{C3} = t_{setup} + 4t_C + t_{mux} = 4.7$	[ns]
(G)	$t_{C4} = t_{setup} + t_C + t_{mux} = 1.7$	[ns]

L'ultimo tempo di carry corrisponde **sempre** a t_{carry} .

Il blocco col propagate deve avere pronto sia il carry precedente che i risultati interni. Ci sono casi in cui il generate è nell'ultimo bit del blocco (come in t_{C4}) e che quindi il carry è subito pronto mentre il blocco deve ancora calcolare le proprie somme. Per questo guardiamo il max.

Il t_{sum} si trova trovando il **carry più lento**. Infatti il blocco successivo dovrà prendere questo carry, farlo uscire (t_{mux}) e poi sommare. dato che prendiamo il t_C più lento i risultati del blocco successivo sono già pronti

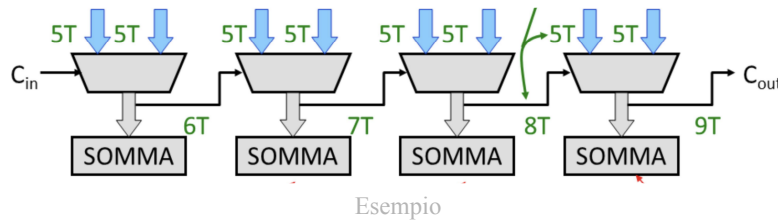
$$t_{sum} = t_{C2} + t_s + t_{mux} = 6.9 \text{ [ns]}$$

Se siamo **nell'ultimo blocco** il t_{mux} **si ignora** poichè il riporto non viene usato.

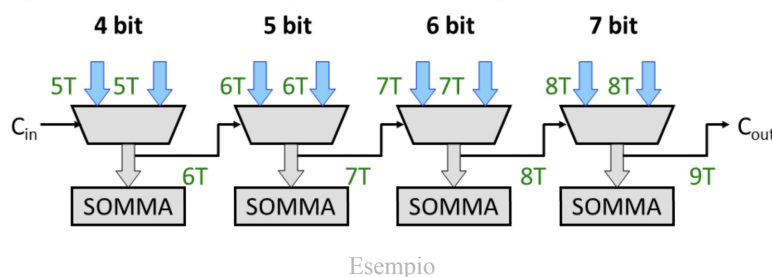
Square-Root Carry Select

Nel caso precedente è facile notare come ogni blocco debba aspettare il MUX del blocco precedente. Quindi i blocchi che finiscono il setup in contemporanea saranno in idle aspettando i MUX. Aumentando la dimensione dei blocchi permette ai blocchi di sfruttare il tempo di idle per calcolare più casi.

Per semplificare i calcoli settiamo $t_c = t_s = t_{mux} = t_{setup} = T$. Con dei blocchi di grandezza $M=4$ è evidente che tutti i blocchi impiegano $t_{setup} + Mt_c = 5T$ a calcolare le uscite e che ogni blocco deve aspettare un T in più per propagare l'uscita.



Si può sfruttare il tempo per calcolare un blocco più grande di 1 bit per volta.



Il numero totale di blocchi sarà

$$N = M + (M + 1) + \dots + (M + K - 1) = K \cdot M + \sum_{i=1}^{K-1} \overset{N \gg M}{\rightarrow} \frac{K^2}{2} \implies K = \sqrt{2N}$$

Analogamente da prima:

$$t_{carry} = t_{setup} + Mt_c + \sqrt{2N} \cdot t_{mux}$$

$$t_{sum} = t_{setup} + Mt_c + \sqrt{2N} \cdot t_{mux} + t_s$$

Lo stesso esempio di prima è applicabile come soluzione generale, **ma stando più attenti** ai coefficienti di t_C .

L'architettura funziona correttamente se $t_C \leq t_{mux}$ ovvero che la il blocco precedente è più lento del tempo t_C del blocco attuale. L'equivalenza da il caso ottimale.

Carry Lookahead

Questo sommatore è il più prestante e anche il più complesso a livello concettuale. Infatti si può dimostrare che il riporto n è dato da C_{in} e tutti i P, G da 0 a n .

$$\begin{aligned} C_0 &= G_0 + P_0 C_{in} \\ C_1 &= G_1 + P_1 C_0 = G_1 + G_0 P_1 + P_1 P_0 C_{in} \\ C_2 &= G_2 + P_2 C_1 = G_2 + G_1 P_2 + G_0 P_2 P_1 + P_2 P_1 P_0 C_{in} \\ &\dots \\ C_n &= \sum_{g=0}^n \left(G_g \cdot \prod_{p=g+1}^n P_p \right) + \left(\prod_{i=0}^n P_i \right) C_{in} \end{aligned}$$

Dove il primo termine è il **segnale generatore di blocco** e il secondo il **segnale propagate di blocco**:
 $C_{out} = G_{0:n} + P_{0:n} C_{in}$

I blocchi possono anche non partire da 0 e non finire a n :

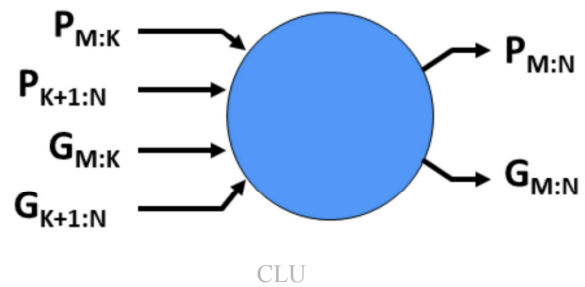
$$\begin{aligned} P_{0:N} &= P_0 P_1 \dots P_N = (P_0 P_1 \dots P_k) (P_{k+1} \dots P_N) = P_{0:k} P_{k+1:N} \\ G_{0:N} &= \sum_{g=k}^n \left(G_g \cdot \prod_{p=g+1}^n P_p \right) + \left(\prod_{i=k}^n P_i \right) \cdot \sum_{g=0}^{k-1} \left(G_g \cdot \prod_{p=g+1}^{k-1} P_p \right) \\ &= G_{k:N} + P_{k:N} G_{0:k-1} \end{aligned}$$

$$\begin{aligned} \underline{G_{0:5}} &= G_5 + G_4 P_5 + G_3 P_4 P_5 + G_2 P_3 P_4 P_5 + G_1 P_2 P_3 P_4 P_5 + G_0 P_1 P_2 P_3 P_4 P_5 = \\ &= G_5 + G_4 P_5 + G_3 P_4 P_5 + P_3 P_4 P_5 (G_2 + G_1 P_2 + G_0 P_1 P_2) = \underline{G_{3:5}} + \underline{P_{3:5} G_{0:2}} \end{aligned}$$

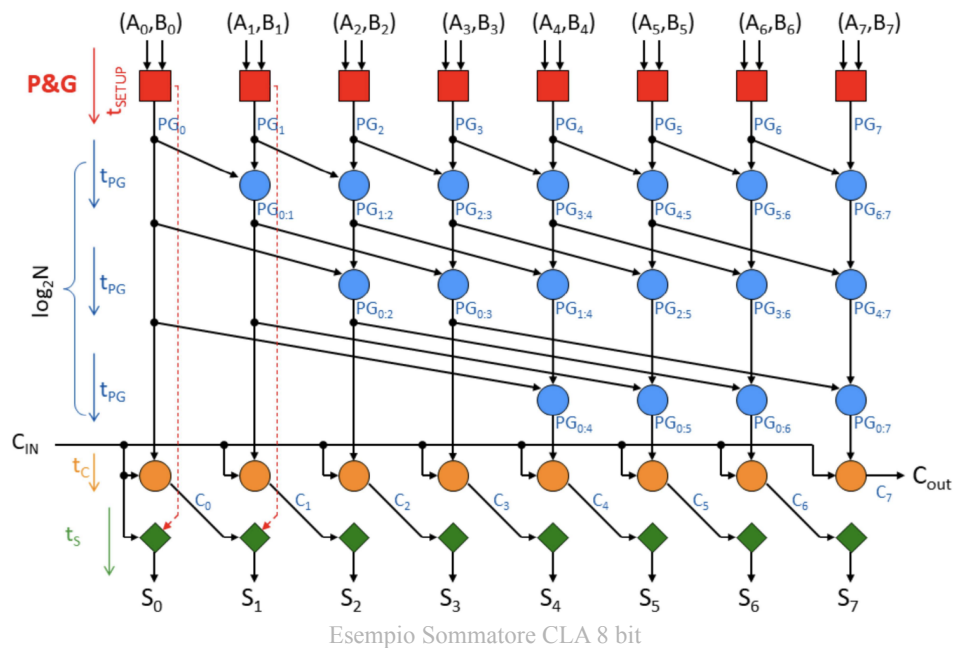
$$\underline{P_{0:5}} = P_0 P_1 P_2 P_3 P_4 P_5 = (P_0 P_1 P_2) (P_3 P_4 P_5) = \underline{P_{0:2} P_{3:5}}$$

Esempio $G(0:5)$

Da qui definiamo l'**Unità Carry Lookahead (CLU)** in grado di Calcolare $P_{M:N}$ e $G_{M:N}$ dati come input due coppie di P e G con estremi $M : K \rightarrow K + 1 : N$



Si possono creare unità più grandi oppure fare delle concatenazioni di unità più piccole. L'implementazione ad albero usa la CLU da 2 coppie in input.



La regola da memorizzare per la costruzione è che partendo dal LSB (sinistra) ad ogni livello L (partendo da 0) questo si conetterà alla CLU del bit 2^L a dx se presente. Arrivati alla fine il risultato andrà usato per calcolare i riporti e infine la somma.

In tutto ci sono i seguenti stadi:

- P e G in parallelo (N blocchi)
- $\log_2 N$ stadi di CLU ($N - 2^{k-1}$ blocchi per stadio $\rightarrow N \log_2 N - N + 1$ in totale)
- Carry (N blocchi)
- Sum (N Blocchi)

$$\# \text{ blocchi in totale: } 2N + N \log_2 N + 1$$

Infine

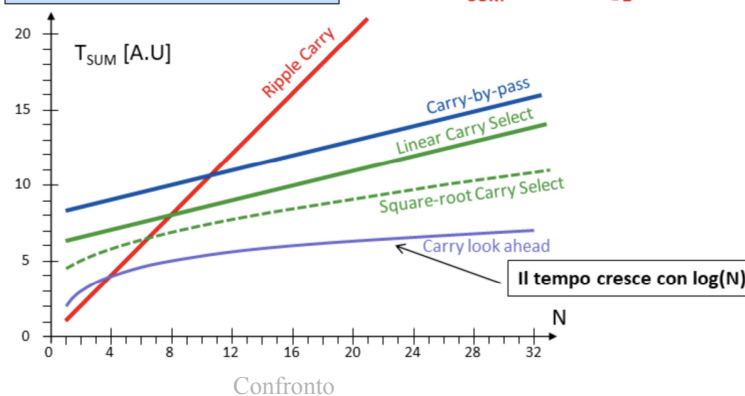
$$t_{carry} = t_{setup} + \log_2 N \cdot t_{PG} + t_c$$

$$t_{sum} = t_{setup} + \log_2 N \cdot t_{PG} + t_c + t_s$$

Architetture a Confronto

Supponiamo per semplicità $t_s = t_c = t_{MUX} = t_{SETUP} = T$

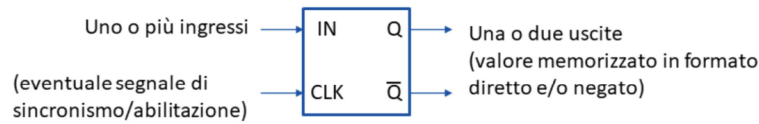
Ripple Carry	$T_{SUM} = (N-1) \cdot t_c + t_s$	$T_{SUM} = N \cdot T$
Carry-Bypass	$T_{SUM} = t_{SETUP} + (2M-1) \cdot t_c + (N/M-1) \cdot t_{MUX} + t_s$	$T_{SUM} = 2M \cdot T + N \cdot (T/M)$
Lin. Carry Sel.	$T_{SUM} = t_{SETUP} + M \cdot t_c + (N/M) \cdot t_{MUX} + t_s$	$T_{SUM} = (M+2) \cdot T + N \cdot (T/M)$
Sqr. Carry Sel.	$T_{SUM} = t_{SETUP} + M \cdot t_c + \sqrt{2N} \cdot t_{MUX} + t_s$	$T_{SUM} = (M+2) \cdot T + \sqrt{2N} \cdot T$
Carry lookahead	$T_{SUM} = t_{SETUP} + \log_2 N \cdot t_{PG} + t_c + t_s$	$T_{SUM} = 3 \cdot T + \log_2 N \cdot T$



7) Latch E Flip-Flop

Nei circuiti sequenziali l'uscita dipende dallo stato successivo, quindi bisogna riuscire ad implementare della **memoria**.

Struttura generale di un latch/FF



Strutture di un latch/FF

L'informazione contenuta è binaria e **subito disponibile all'uscita**. Inoltre il tipo di scrittura può essere **sincrono o asincrono** in base alla sincronizzazione con un clock.

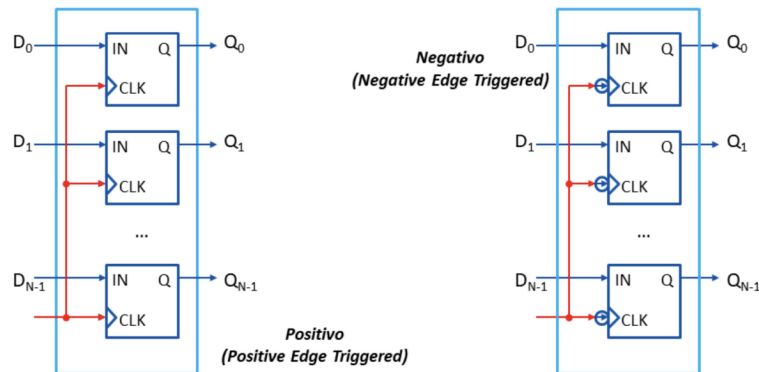
Un **Latch** è sensibile ai *livelli* alti e bassi del clock.

- Quando è **abilitato** si dice che è **trasparente**: i valori in entrata e uscita sono gli stessi
- Quando è **disabilitato** si dice **negativo**: il valore in uscita è l'ultimo valore assunto, indipendentemente dall'entrata

Un latch è positivo se abilitato con CLK=1, altrimenti negativo.

Registri

Con i FF si possono costruire registri a n-bit mettendo insieme di n FF sincronizzati con lo stesso clock:

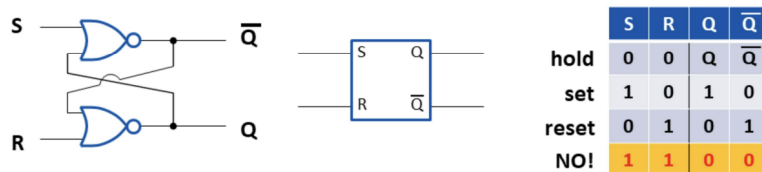


Registro

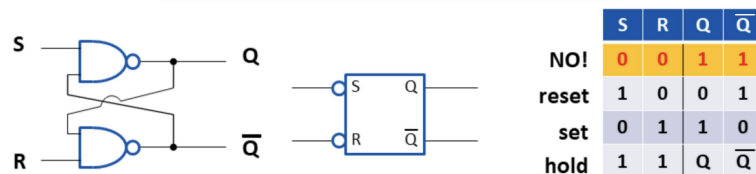
7.2) Implementazione Logica Latch

Iniziamo con dei latch asincroni:

Latch SR con porte NOR (cross coupled NOR)



Latch-SR con porte NAND (cross coupled NAND)



Latch SR Asincrono

Per svolgere l'analisi definiamo Q_n lo stato attuale e Q_{n-1} quello precedente

$$Q_n = R + \overline{Q_{n-1}}$$

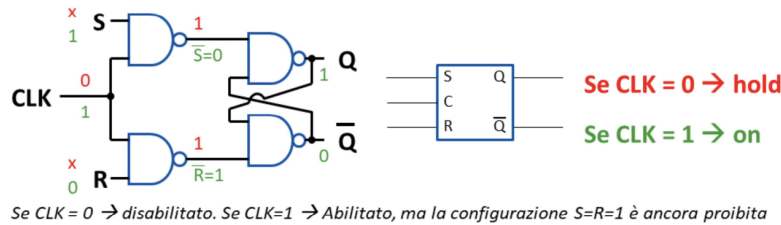
$$\overline{Q_{n-1}} = \overline{S + Q_{n-1}}$$

è importante notare che $S=R=1$ è proibita poichè rende instabile il circuito e forzerà una delle due uscite ad 1.

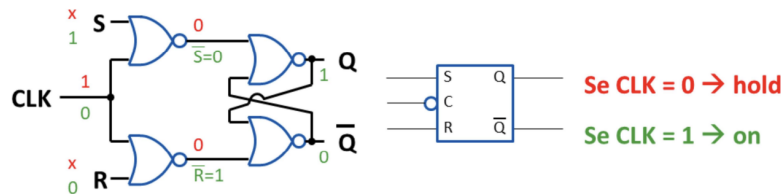
- **Hold**, $S=R=0$: $Q_n = Q_{n-1}$
- **Set**, $S=1$, $R=0$: $Q_n = 1$
- **Reset**, $S=0$, $R=1$: $Q_n = 0$

Per costruire un latch sensibile ai livelli si utilizzano latch asincroni con entrate date dal clock e S/R in delle porte logiche.

Latch-SR sensibile ai livelli (positivo)



Latch-SR sensibile ai livelli (negativo)



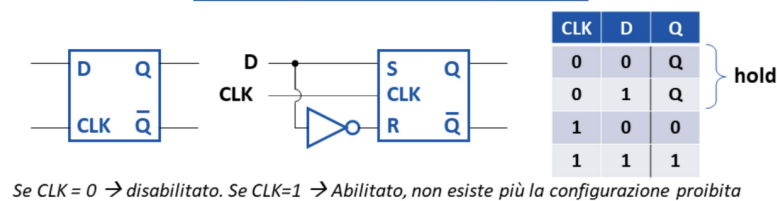
Latch SR Sincroni

In questo caso le condizioni di hold/set/reset sono ad entrate invertite.

Nel caso positivo la porta NAND forza $S=R=1$ (hold) del latch asincrono quando il clock è basso. Con il clock alto le porte dipenderanno dal valore di S e R.

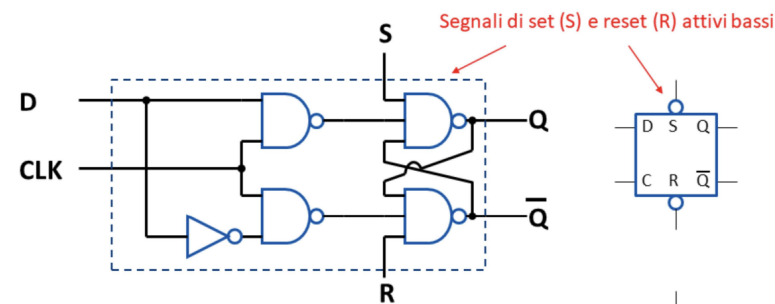
Infine si configura il latch D. In questa configurazione $D = S = \overline{R}$.

Latch-D sensibile ai livelli (positivo)



Latch D

Modificando il latch SR asincrono con porte a 3 ingressi si possono forzare S e R asincroni dal clock per forzare valori di Q a 0 o 1 indefinitivamente



Latch D sincrono con SR asincrono

Queste porte sono usate per settare dei valori iniziali o disattivare dei registri

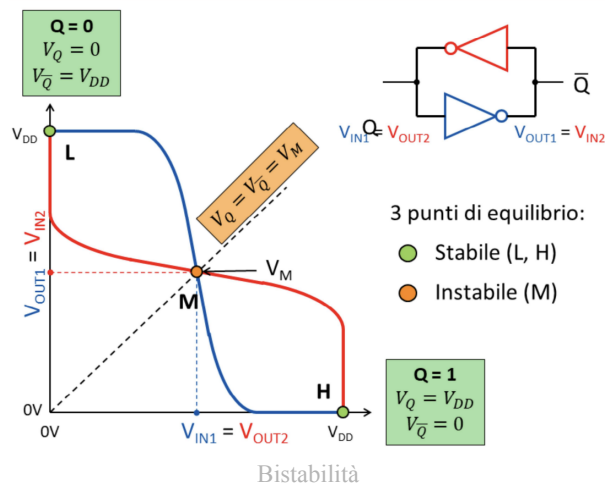
- $S=0, R=1$: $Q=1$
- $S=1, R=0$: $Q=1$
- $S=R=0$: il latch funziona normalmente

7.2.1) Implementazione Ottimizzata

I circuiti visti in precedenza usano 8-16-18 transistor. Queste porte non sono utilizzate in implementazioni reali poichè ne esistono di più ottimizzate.

Principio di Bistabilità

Il principio di bistabilità consiste nello sfruttare una retroazione positiva per trovare una V_m con 2 punti stabili e 1 instabile.



La curva rossa rappresenta $V_{o2} - V_{i2}$. Da qui notiamo il principio di bistabilità che dice: **Se il guadagno dell'invertitore è maggiore di 1 nella regione centrale della caratteristica, i punti esterni sono stabili e quello intermedio è instabile**

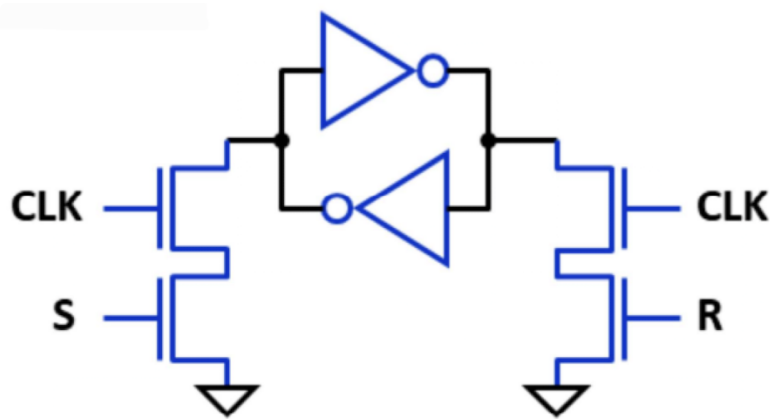
✓ Proof

Questo principio permette anche di implementare la proprietà rigenerative (L e H sono immuni ai disturbi con ampiezza minore di $\min\{V_M, V_{DD} - V_M\}$)

Finora tutti gli esempi sono essenzialmente bistabili.

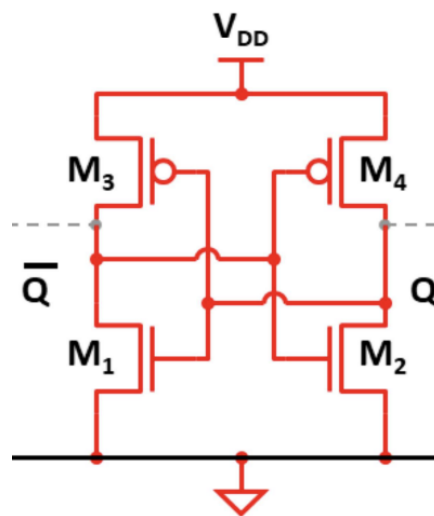
Scrittura per Forza Bruta

Per cambiare il valore memorizzato si può utilizzare un **feedback overpowering**, cioè forzare il nuovo valore di Q ad un livello oltre V_M così da forzare il punto di stabilità opposto. Con un Pass-T è importante che $R_{switch} \ll R_{not}$



Asincrono

Si usano 6 MOS per la versione asincrona, 8 per la versione sincrona (aggiunta CLK) e 2 in più nel caso si dovesse fare un Latch D (invertitore in più)



Insight al circuito bistabile

La **fase di memoria** avviene con CLK=0. Qui M_6, M_8 sono off.

La **fase di scrittura** avviene con CLK=1. Qui M_6, M_8 sono on

$$Q = \overline{CLK \cdot R + \overline{Q}}$$

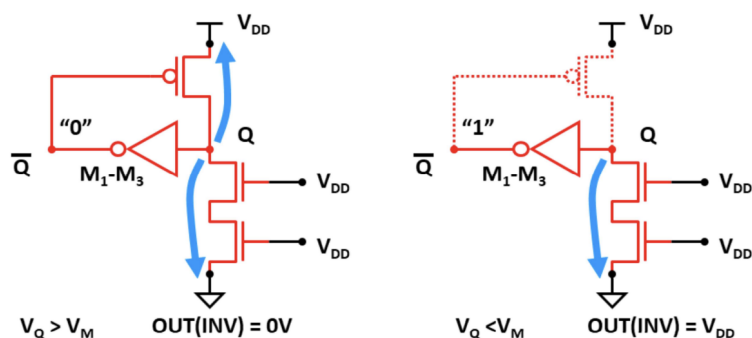
$$\overline{Q} = \overline{CLK \cdot S + \overline{Q}}$$

Se S=R=0 M_5, M_7 sono OFF e quindi rimane comunque la fase di memoria

Se R=1 M_7 è ON e V_Q diminuisce facendo spegnere Q_2, Q_3

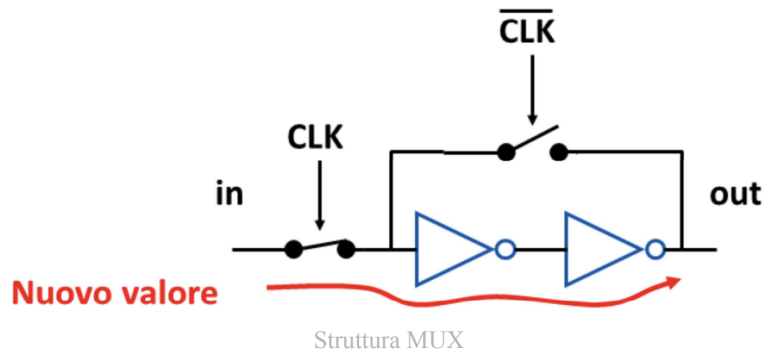
Condizione di scrivibilità:

Per scrivere "0" nel nodo Q è necessario che M_7, M_8 siano in grado di far scendere la tensione di Q fino alla soglia logica V_M degli invertitori, tutto questo in mezzo ciclo di clock. Quindi il loro dimensionamento deve essere grande.

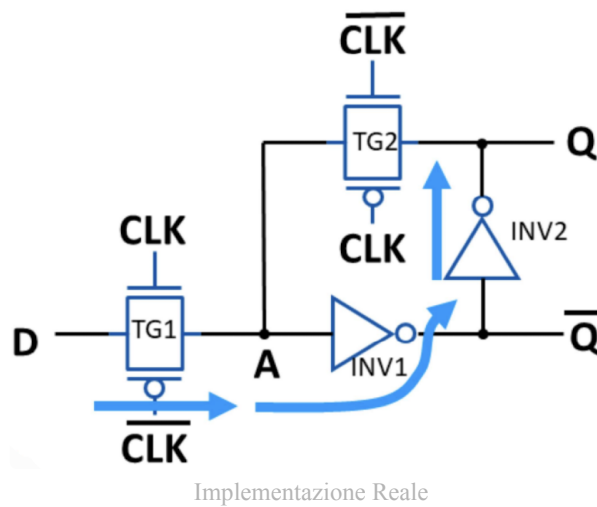


Struttura MUX

Un secondo metodo è la **struttura MUX** e consiste nell'interrompere il feedback positivo. Qui il risultato è indipendente dalle resistenze:



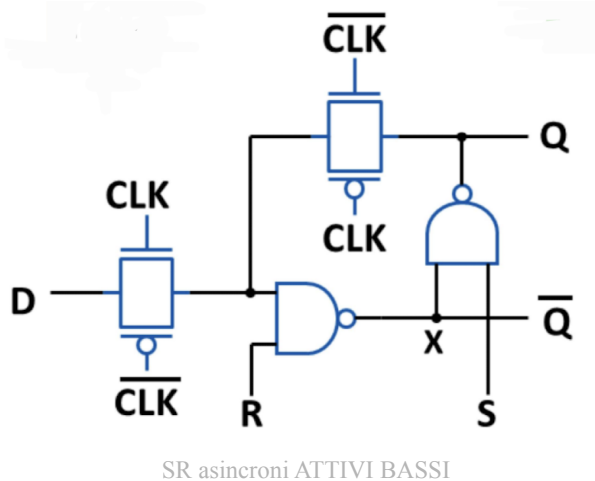
In questo caso $OUT = CLK \cdot IN + \overline{CLK} \cdot OUT$



Si usano **8 MOS** in totale

In questo caso il **tempo di memorizzazione** dipende dal tempo di propagazione dei 2 invertitori e di un TG in quanto A e Q devono avere lo stesso potenziale

Si possono anche aggiungere segnali di set/reset asincroni:



Per un implementazione **attiva alta** si cambiano i NAND coi NOR, e invertendo S e R

$$X = \overline{S + Q}$$

$$Q = \overline{R + X}$$

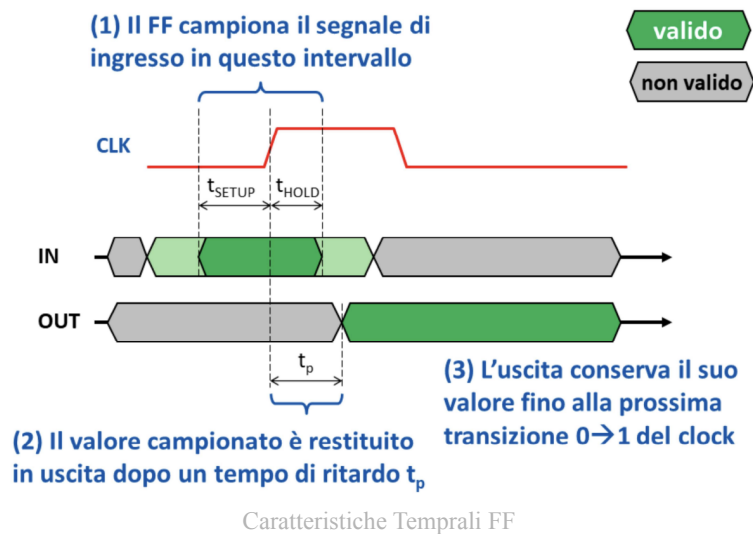
- S=R=0. NOR diventano invertitore, il latch funziona normalmente
- S=0, R=1: $X = \overline{Q} \rightarrow Q = 0 \rightarrow X = 1$ Reset asincrono
- S=1, R=0: $X = 0 \rightarrow Q = \overline{Q} = 1$ Set asincrono

7.3) Flip Flop

Il flip flop è sensibile ai *fronti* del clock: il campionamento avviene ogni Positive o Negative edge, da qui i nomi PET o NET (positive/negative Edge Trigger).

Caratteristiche Temporal FF

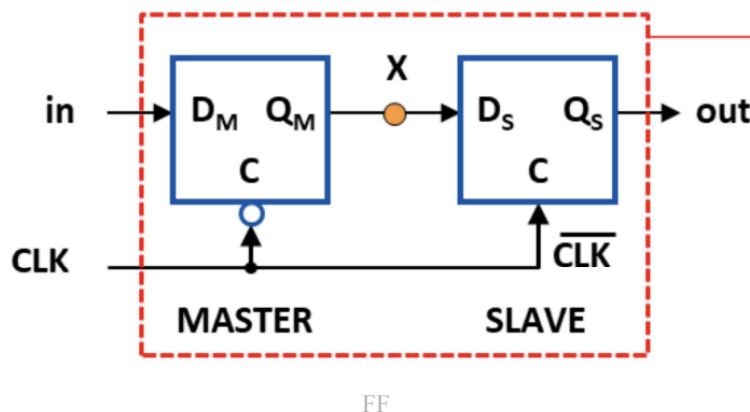
Negli intorno della commutazione il FF è detto sensibile in quanto per memorizzare correttamente il dato l'ingresso deve essere stabile



- t_{setup} intervallo prima del clock dove FF è trasparente
 - t_{hold} intervallo dopo la commutazione dove FF è trasparente
 - t_p classico tempo di propagazione
- La somma $t_s + t_h$ è il tempo che FF impiega per immagazzinare

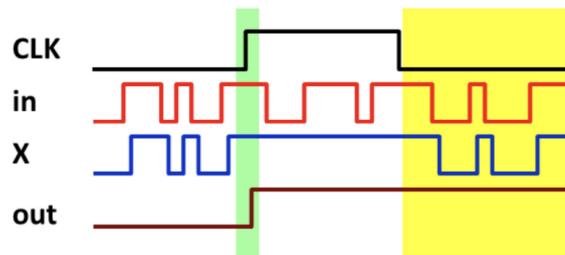
Master-Slave

Viene costruito con 2 latch opposti.



I due latch sono attivi in periodi opposti del clock e quindi l'uscita cambierà solo al rising/falling edge. Negli altri momenti è in fase di memoria. (in questo caso RET)

Con CLK=0 il master è on e campiona l'entrata, con CLK=1 il master è off e lo slave ON quindi l'uscita sarà data dal campione del master



Esempio

Analizziamo i tempi:

$$t_p^{MASTER} = t_p^{TG1} + t_p^{INV1} + t_p^{INV2}$$

$$t_p^{SLAVE} = t_p^{TG3} + t_p^{INV3} + t_p^{INV4}$$

$$T_{min} = t_p^{MASTER} + t_p^{SLAVE} \rightarrow f_{Max} = 1/T_{min}$$

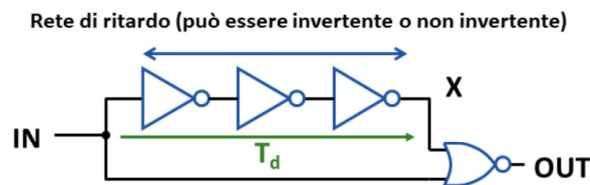
Remark.

Nel tempo di propagazione, ovvero il tempo per passare dal primo al secondo latch, non si andrà a considerare la capacità del nodo in cui i latch si uniscono in quanto quella capacità avrà avuto tempo di caricarsi/scaricarsi prima del cambio del clock che inverte gli stati ON/OFF dei due latch

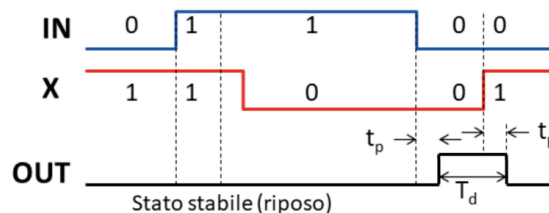
FF ad Impulsi

Con un clock ad impulsi, con lunghezza dell'impulso abbastanza grande per campionare l'entrata è possibile costruire un flip flop. In questo caso il latch campiona solo in un intorno del fronte di salita e per ciò l'impulso deve essere abbastanza lungo per permettere di memorizzare correttamente il dato.

In questo caso il CLK va a pilotare il generatore ad impulsi



Generatore di Impulsi



Segnali

L'idea dietro al circuito è molto semplice. Il segnale X sarà ritardato rispetto ad IN. Quindi disegnando un ciclo di clock è possibile poi compilare la tabella di verità dell'uscita per capire che porta usare in uscita.

Remark.

Bisogna scrivere l'uscita in modo tale che la porta sia pilotata da CLK e non il suo negato.

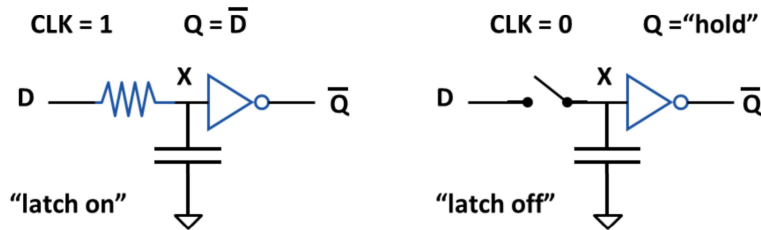
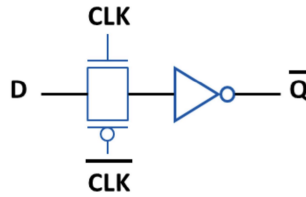
Se invece si sceglie un numero N di invertitori e risulta che l'entrata della porta è data da X negato allora si

dovranno usare $N+1$ invertitori. Una volta trovato se il numero di invertitori è pari o dispari si può trovare il numero totale di invertitori facendo $M = t_{desiderato}/t_p$

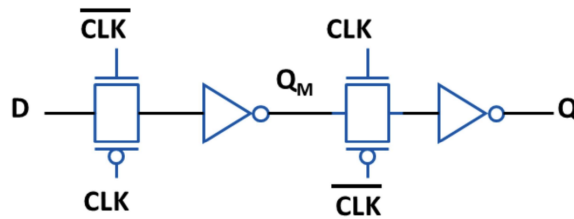
7.4) Implementazione Dinamica

In questo caso l'informazione è tenuta nella carica Q di una capacità C che verrà caricata a 0 o V_{DD} . Tuttavia l'informazione è temporanea perchè la capacità tenderà a scaricarsi.

4 MOS ($2n + 2p$)



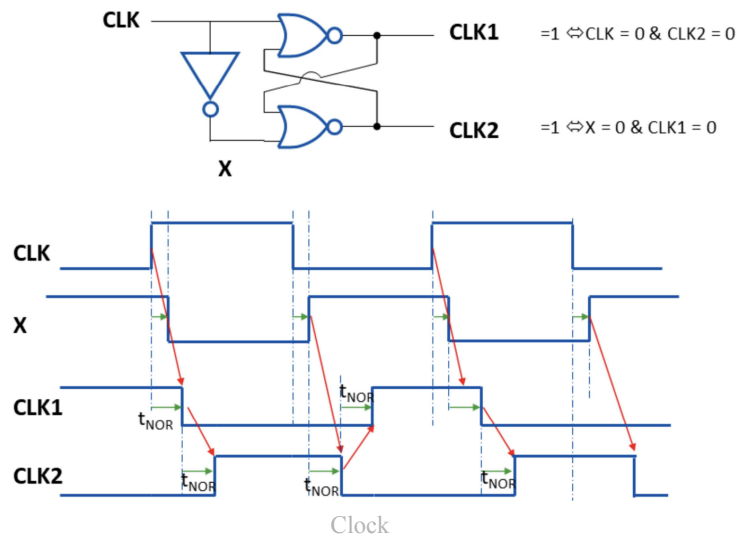
Latch Dinamico



FF Dinamico

7.5) Clock Reale

Dato che per un FF serve un CLK negato ci sarà un delay tra CLK e $\overline{CLK}$. Questo porta una sovrapposizione degli stati 00 e 11 dei due latch che può causare una memorizzazione sbagliata dei dati



Clock

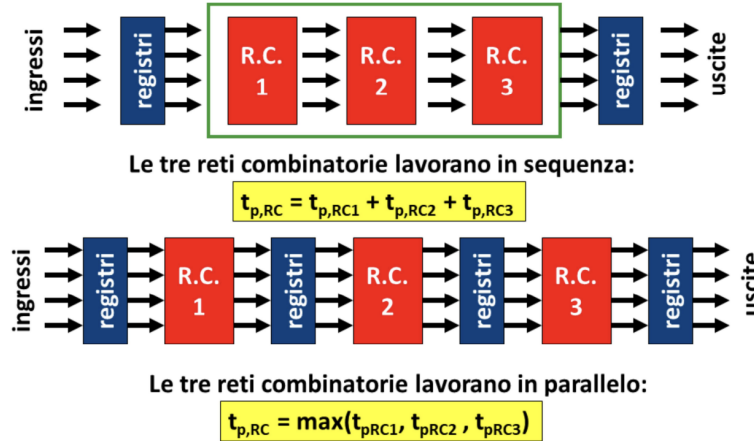
Il clock in figura permette di non avere mai nessuna sovrapposizione 11

8) Circuit Sequenziali

Prima di parlare dei circuiti sequenziali veri e propri parliamo dell'architettura pipeline, che sono l'ultimo argomento del capitolo precedente

8.1) Architettura Pipeline

L'architettura pipeline è un metodo per migliorare le prestazioni di una rete combinatoria tradizionale introducendo un parallelismo tra i vari stadi.



Esempio

In questi circuiti andiamo a definire due tempi determinati dai FF e dalle reti combinatorie

- **Tempo di propagazione:** è il tempo affinché la rete più lenta si propaga da un FF al successivo

$$T_{prop} = t_{p,FF} + \max_i(t_{p,rete\ i\ esima})$$

- **Tempo di contaminazione:** è il tempo minimo per raggiungere il FF e sovrascrivere il vecchio dato, ovvero

$$T_{cont} = t_{p,FF} + \min_i(t_{p,rete\ i\ esima})$$

Infine è importante che $T_{min} = T_{prop} + t_{setup} < T_{CLK}$ così da dare tempo a tutti i dati di salvarsi correttamente nei FF. Da qui è evidente che la frequenza massima è data da

$$f_{max} = \frac{1}{T_{min}} = \frac{1}{T_{prop} + t_{setup}}$$

Inoltre se il tempo di hold è diverso da 0 dobbiamo anche verificare la seguente condizione

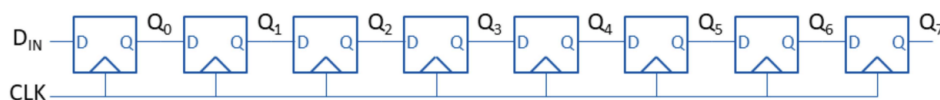
$$T_{cont} > t_{hold}$$

Più stadi si creano, più aumenta la latenza di un ciclo di clock per stadio

8.2) Registro a Scorrimento

Il registro a scorrimento è un semplice circuito sequenziale ad un entrata, N bit di stato (2^N stati) ed N uscite

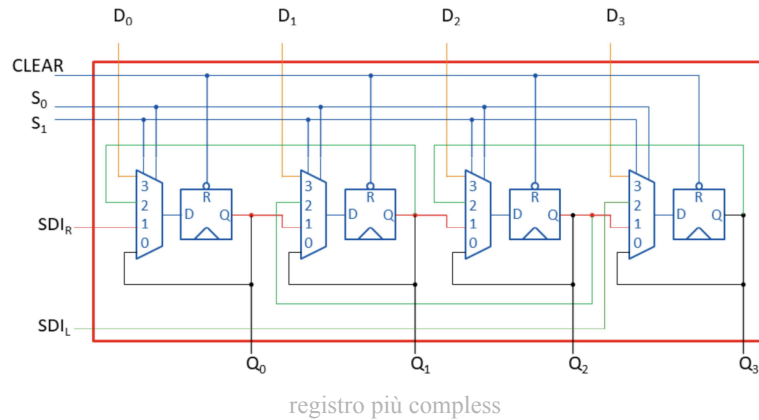
$$Q_k^{(n+1)} = \begin{cases} D_{IN}^{(n)} & k = 0 \\ Q_{k-1}^{(n)} & k \in [1, N-1] \end{cases}$$



Esempio 8 bit

implementazioni più complesse permettono di avere Reset asincrono, caricare una sequenza specifica e anche di

selezionare la direzione di scorrimento



In questo caso i bit S_1S_0 indicano la funzione che svolge il circuito

- 00: il registro mantiene caricati gli stessi valori
- 01: scorrimento a destra (funzione normale)
- 10: scorrimento a sinistra
- 11: memorizzazione in parallelo della sequenza D

8.3) Contatore

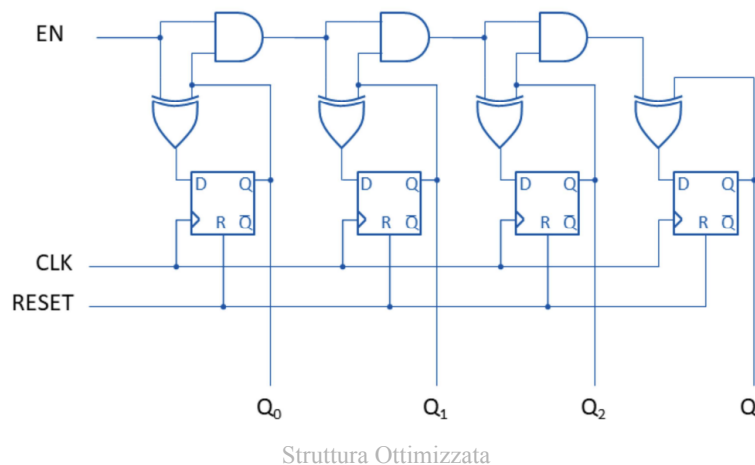
Il contatore è un circuito sequenziale a 0 entrate, 2^N stati e N uscite. Questo andrà ad aumentare di 1 il valore binario salvato fino al suo reset una volta raggiunto il valore massimo $\dots \rightarrow 2^N - 1 \rightarrow 0 \rightarrow \dots$

La struttura logica del contatore può essere generalizzata nel seguente modo:

$$\begin{aligned} Q_0^{(n+1)} &= \overline{Q_0^{(n)}} \\ Q_1^{(n+1)} &= Q_1^{(n)} \oplus Q_0^{(n)} \\ Q_2^{(n+1)} &= Q_2^{(n)} \oplus (Q_1^{(n)} \cdot Q_0^{(n)}) \\ &\dots \\ Q_k^{(n+1)} &= Q_k^{(n)} \oplus (Q_{k-1}^{(n)} \cdot \dots \cdot Q_0^{(n)}) \end{aligned}$$

Inoltre in un'implementazione reale c'è un ulteriore input EN che può essere visto come un'entrata di tipo Q_{-1} settato asincronamente

- $EN = 0$: il contatore è disabilitato $Q_k^{(n+1)} = Q_k^{(n)}$ dato che $Q_0^{n+1} = Q_0^n \oplus EN = Q_0^n$
- $EN = 1$: il contatore è abilitato dato che $Q_0^{n+1} = Q_0^n \oplus EN = \overline{Q_0^n}$

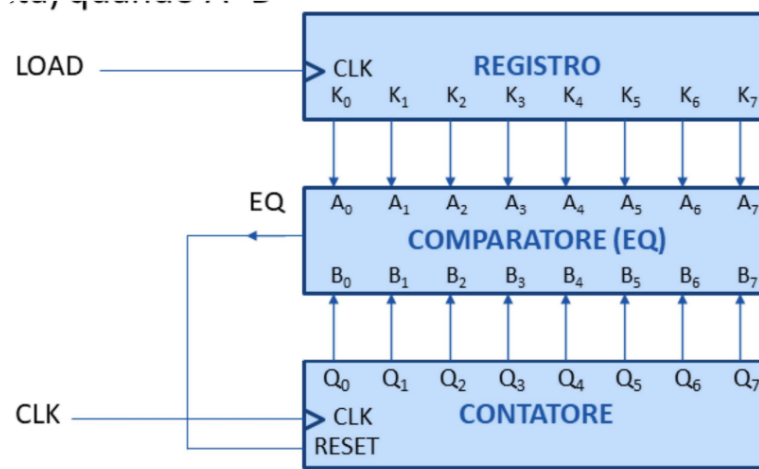


è chiaro che il contatore non è ancora completo, in quanto manca il reset una volta raggiunto il valore massimo.

Definition 5 (Modulo di un contatore).

Chiamiamo K il modulo del contatore come il numero di cicli dopo cui il contatore torna allo stato iniziale. In particolare abbiamo che $K \leq 2^n - 1$

Per farlo verrà usato un comparatore collegato alle uscite a alle uscite di un registro contenente il valore del modulo.

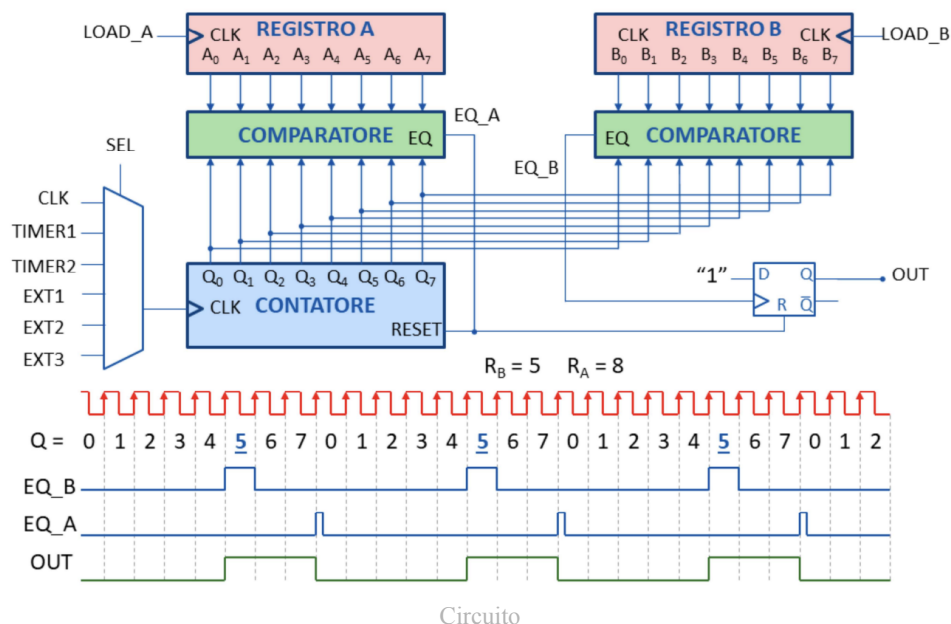


Esempio

Quando il comparatore e il contatore sono allo stesso valore, il comparatore avrà uscita 1 che essendo collegata al reset del contatore porterà tutti i valori a 0 (stato iniziale)

8.4) Timer/Generatore di Forme d'Onda

Come vedremo, questo timer sarà un divisore di frequenza con duty time squilibrato



Circuito

It is important to notice that $R_A > R_B$ since the comparator B acts as the rising edge of the output FF. Then the comparator B resets the FF and the counter.

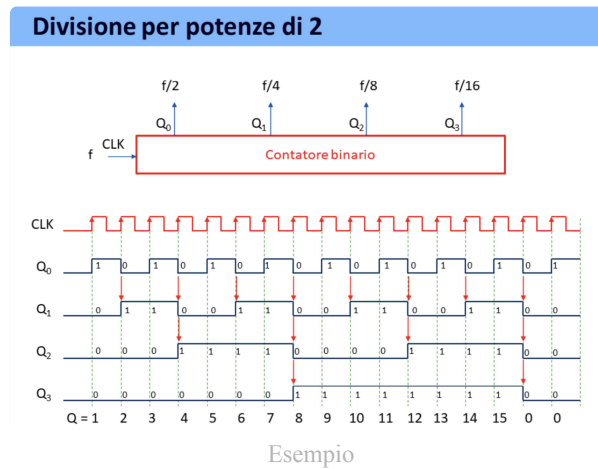
$$f_{out} = f_{in}/R_A$$

$$\text{Duty cycle: } (R_A - R_B)/R_A$$

8.5) Divisore di Frequenza

8.5.1) Potenze di 2

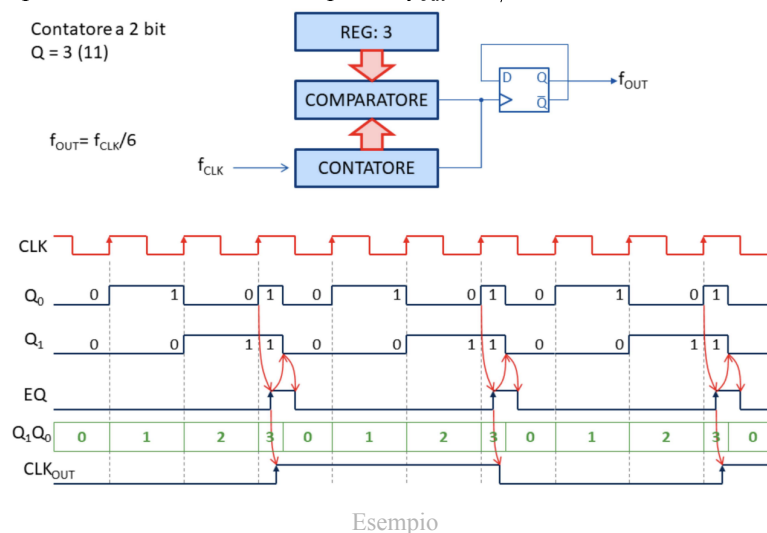
Essenzialmente in un contatore ogni bit cambia con frequenza $f/2^k$. Basta selezionare l'uscita dal contatore selezionato:



Per calcolare la potenza dinamica in questo caso è facile notare che in ogni stadio (a parità di capacità) nel calcolo verrà usata la frequenza divisa

8.5.2) Numero Arbitrario Pari

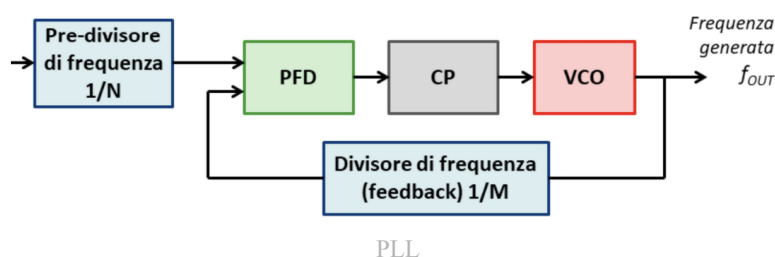
In questa implementazione si usa l'output di un EQ come clock di un FF retroazionato. Se il contatore ha modulo K e entrata f_{in} , allora l'output del FF oscillerà con frequenza $f_{out} = 1/2K$



8.5.3) Numero Arbitrario Razionale (Phase Locked Loop) (PLL)

Questo complesso circuito sequenziale misto digitale analogico retroattivo permette di avere una sequenza moltiplicata per un numero arbitrario deciso da due fattori di divisione di un circuito divisore a numeri pari:

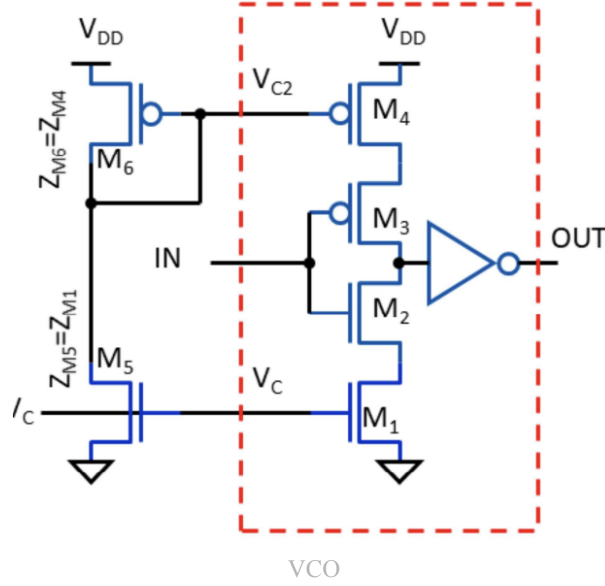
$$\frac{f_{in}}{N} = \frac{f_{out}}{M} \Rightarrow f_{out} = \frac{M}{N} f_{in}$$



Andiamo a studiare i 3 componenti mancanti:

Voltage Controlled Oscillator (VCO)

Come suggerisce il nome questo è un oscillatore che invece di essere controllato in voltaggio (non modificabile) viene pilotato in corrente. Infatti tutti i componenti dell'oscillatore (tranne l'ultimo NOT) sono pilotati in corrente, che a sua volta è determinata da un potenziale V_C non collegato a V_{DD} ma determinato dalla CP. Questi invertitori a controllo di corrente non sono realmente invertitori in quanto negano due volte l'input, da qui la necessità dell'ultimo not (oscillatore ha N dispari).



- M_2, M_3 sono interruttori pilotati da IN. Invertitore #1
- M_1 generatore di corrente direttamente pilotato da V_C
- M_5, M_6 specchio di corrente
- M_4 generatore di corrente pilotato da V_{C2}

Chiamo $Z = Z_i, \forall i \in [1, 6]$ il fattore di forma uguale a tutti i MOS.

Se V_C è abbastanza piccola da tenere in saturazione M_1, M_5 questi saranno generatori di corrente:

$$I_{1,5} = \frac{1}{2} k'_n Z (V_C - V_{tn})^2$$

Dallo specchio di corrente sappiamo che la corrente a M_6 è la stessa di M_5 e da qui possiamo trovare V_{C2} :

$$\begin{cases} I = \frac{1}{2} k'_p Z_6 (V_{C2} - V_{DD} - V_{tp})^2 \\ I = \frac{1}{2} k'_n Z_5 (V_C - V_{tn})^2 \end{cases} \implies V_{C2} = V_{DD} + V_{tp} - \sqrt{\frac{k'_n Z_5}{k'_p Z_6}} (V_C - V_{tn})$$

Da qui si verifica anche che con V_{C2} sufficientemente piccola da tenere M_4 in saturazione allora

$$I = \frac{1}{2} k'_p Z (V_{C2} - V_{DD} - V_{tp})^2$$

E quindi tutte le correnti saranno uguali:

$$I = \frac{1}{2} k'_n Z (V_C - V_{tn})^2 = \frac{1}{2} k'_p Z (V_{C2} - V_{DD} - V_{tp})^2$$

$$V_{C2} = V_{DD} + V_{tp} - \sqrt{\frac{k'_n Z_5}{k'_p Z_6}} (V_C - V_{tn})$$

Dalla funzione caratteristica della capacità si trova il tempo di carica/scarica

$$\frac{dV_x}{dt} = \frac{I}{C} \rightarrow \begin{cases} V_x = V_{DD} - \frac{I \cdot t}{C} & \text{se scarica} \\ V_x = \frac{I \cdot t}{C} & \text{se carica} \end{cases}$$

Dato che t_p è definito come il tempo per caricare/scaricare M_3, M_2 fino a $V_{DD}/2$ basta sostituirlo a V_x per trovare Δt :

$$\Delta t = \frac{C \cdot V_{DD}}{2 \cdot I} = \frac{C \cdot V_{DD}}{k'_n Z (V_C - V_{tn})^2} \rightarrow t_p = t_{INV} + \Delta t \approx \Delta t$$

Quindi collegando N invertitori a corrente + 1 invertitore troviamo che il tempo di propagazione dell'intero anello è:

$$T_p = N \cdot \Delta t + (N + 1)t_{INV} \approx N \frac{C \cdot V_{DD}}{k'_n Z (V_C - V_{tn})^2} \rightarrow F = \frac{1}{2T_p} \frac{k'_n Z (V_C - V_{tn})^2}{2N \cdot C \cdot V_{DD}}$$

qui si nota come la corrente sia monotona crescente rispetto a V_C

Comparatore di Fase e Frequenza (PFD)

Il Phase Frequency Detector (PFD) è un circuito a stati, ovvero che l'output cambia all'avvenire di alcuni eventi (asincrono ma anche no, poi capisci)

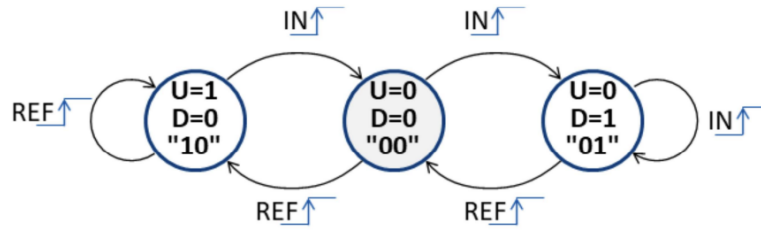
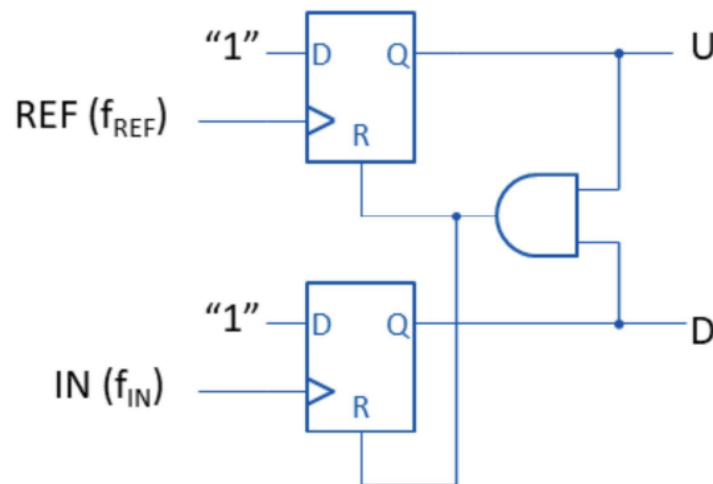


Diagramma di Stati

I bit di uscita sono 2 quindi ci sono 4 stati possibili, i bit in ordine sono scritti U, D che mnemonicamente indicano Up e Down, cioè cosa deve fare il CP alla fase di IN

- 11: questo è lo stato vietato, ovvero che non indica nessuna informazione utile. Vedremo che questo porta subito al reset di entrambe le porte 11 $\rightarrow$ 00
- 00: stessa frequenza e stessa fase
- $D = 0, U$ oscilla: IN ha frequenza maggiore di REF, oppure è in ritardo su REF (con stessa frequenza)
- D oscilla, $U = 0$: IN ha frequenza minore di REF, oppure è in anticipo su REF (con stessa frequenza)

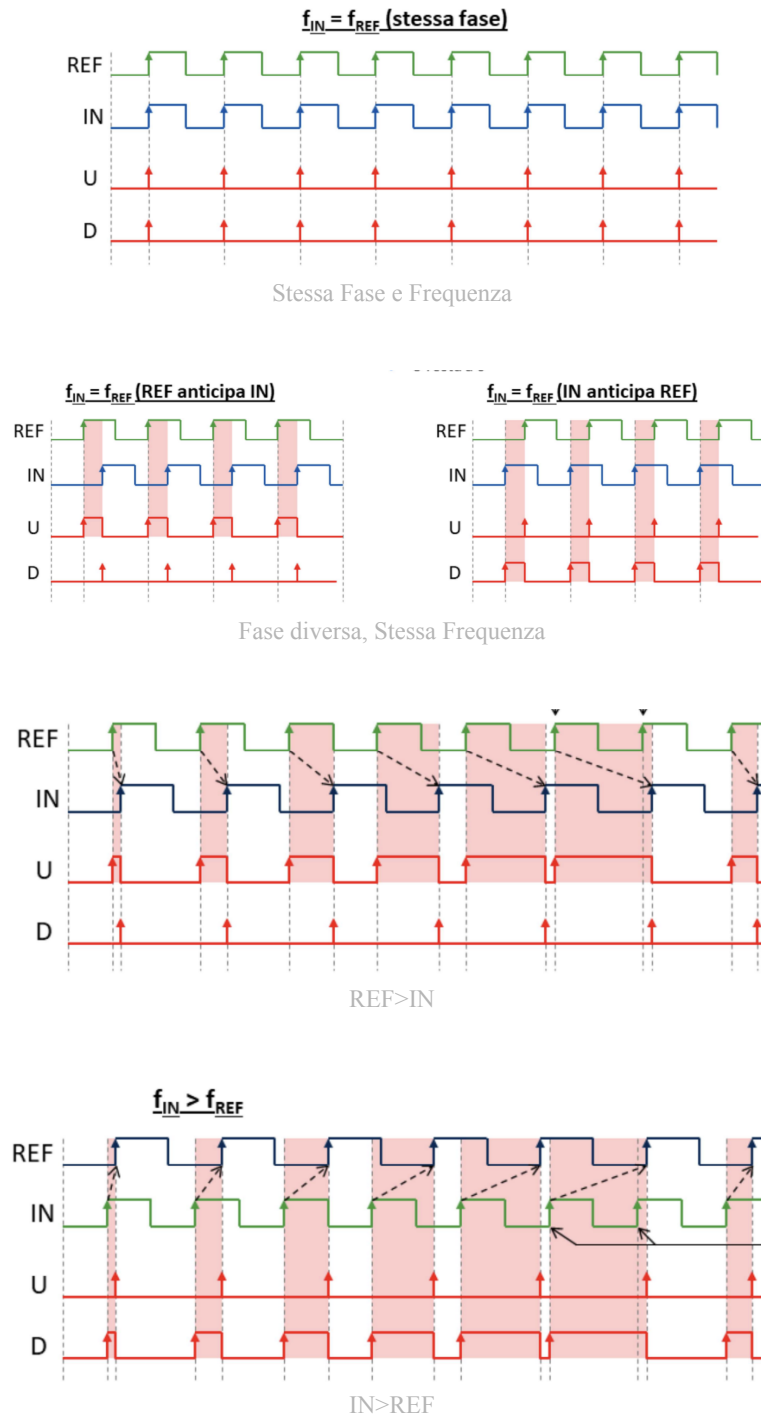


Circuito

I FF sono pilotati dal rising edge delle due frequenze e scriveranno il dato 1. è chiaro come partendo da "00" il primo rising edge vada a determinare lo stato successivo. Quello in anticipo o il più veloce andrà a far oscillare il suo bit

REF in anticipo/più veloce → U oscilla
 IN in anticipo/più veloce → D oscilla

La scrittura di 0 per ogni FF avviene solo grazie al reset asincrono dettato dalla porta AND. Infatti a prescindere dallo stato, se uno dei due bit è ad 1, nel momento in cui il anche l'altro va ad 1 la porta AND provocherà il reset. Qui il diagramma delle frequenze nei vari casi:

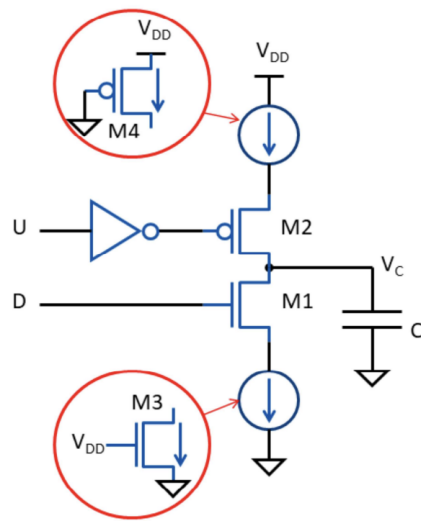


Negli ultimi due casi è evidente come la differenza di frequenza non provochi problemi anche se ci sono due fronti consecutivi

Pompa di Carica (CP)

LA pompa di carica è semplicemente un circuito con MOS pilotati da U e D del PFD così da fornire al VCO dei valori di V_C variabili per arrivare ad equilibrio.

- U oscilla: Bisogna aumentare la frequenza, V_C si collega a V_{DD} ed aumenta
- D oscilla: Bisogna diminuire la frequenza, V_C si collega a massa e diminuisce

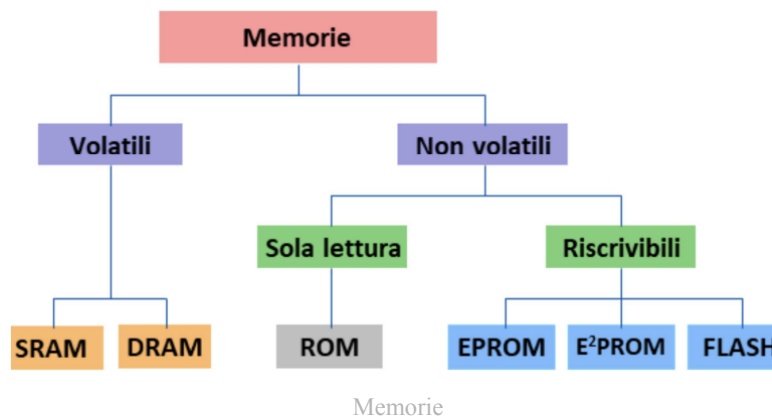


Circuito

Z_3, Z_4 devono essere abbastanza piccoli per variare lentamente V_C così da garantire la stabilità della retroazione.

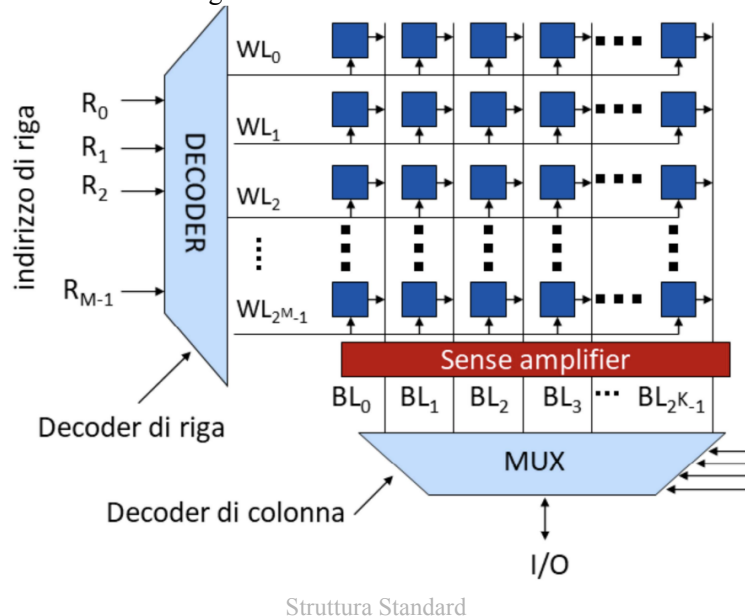
9) Memorie a Semiconduttore

In questo capitolo studieremo le memorie. L'immagine seguente mostra come sono classificate i vari tipi di memoria.



I FF non possono essere usati come memoria a larga scala in quanto impiegherebbero centinaia miliardi di transistori per un solo gigabyte. ($16 \cdot 8 \cdot 2^{30} = 128$ miliardi)

Per implementare una memoria si usa la seguente struttura



Questo banco tiene 2^N celle con $N = M + K$.

- 2^M righe chiamate **wordline**
- 2^K colonne chiamate **bitline**

Vari blocchi possono essere mergati da un selettore di blocco che permette di avere $N = M + K + B$.

Il MUX svolge lo stesso lavoro del DECODER con la capacità aggiuntiva di poter indirizzare la direzione dell'informazione (R/W oppure I/O)

9.1) Capacità e Resistenze delle WL e BL

Prima di studiare le memorie è importante notare che le BL e WL, essendo conduttori molto grandi, presentano resistenze e capacità parassite non trascurabili. Useremo infine Elmore per fare un modello dei tempi di ritardo

Data una WL di larghezza L , lunghezza D e altezza X è possibile trovarne la resistenza della cella N partendo dalla resistenza del quadro $R_q = \rho/X$.

$$R = R_q \frac{D}{X} \rightarrow R_{WL} = NR$$

La capacità parassita di una cella è data da

$$C = C_{in} + C_{area}LD + C_{bordo}2D \rightarrow C_{WL} = NC$$

E infine il modello di Elmore darà il seguente risultato

$$t_p = 0.69R_{dec}NC + 0.69RC\frac{N^2}{2} = 0.69R_{dec}C_{WL} + 0.69\frac{R_{WL}C_{WL}}{2} = 0.69C_{WL}(R_{dec} + \frac{R_{WL}}{2})$$

	WL	BL
R	$R_q \frac{D}{L}$	$R_q \frac{H}{L}$
C	$C_{in} + C_{area}DL + C_{bordo}2D$	$C_{in} + C_{area}HL + C_{bordo}2H$
t_p	$0.69R_{dec}C_{WL} + 0.69\frac{R_{WL}C_{WL}}{2}$	$0.69R_{dec}C_{BL} + 0.69\frac{R_{BL}C_{BL}}{2}$

Remark.

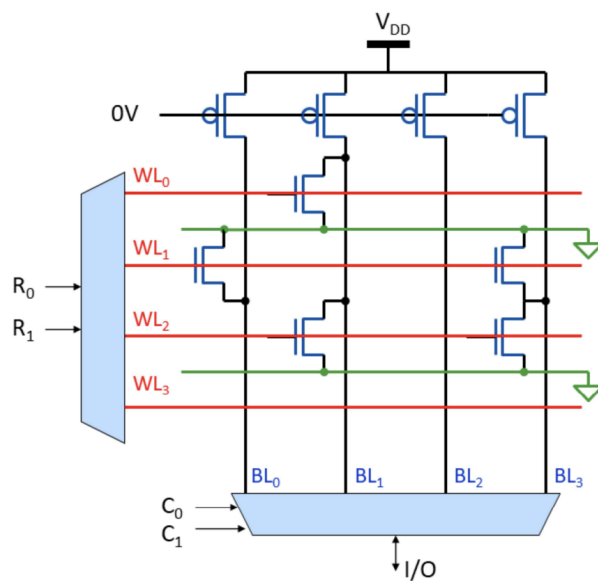
Nella memoria NOR a maschera CONTACT il modello di t_p non è applicabile in quanto C non è comune a tutte le celle. Una cella che salva 0 avrà $C_{in} = C_g$, mentre le altre avranno $C_{in} = 0$

Nella memoria NAND a maschera METAL1 invece i CC daranno una resistenza nulla, ma capacità presente

9.2) Read Only Memory (ROM)

NOR

Come già spiegato ad inizio capitolo e come ricordato dal nome, questa memoria viene costruita con già il codice embedded in essa, ovvero che viene costruita con le connessioni desiderate che non possono essere cambiate.



Esempio ROM

WL_M permette di scegliere la riga in cui accendere il MOS (se presente). Il MOS connesso a ground infatti permette di mettere il potenziale di BL_K ad un valore V_{OL} basso vicino a 0

Guardando la funzione logica di una bitline è facile capire il senso del nome NOR, per esempio

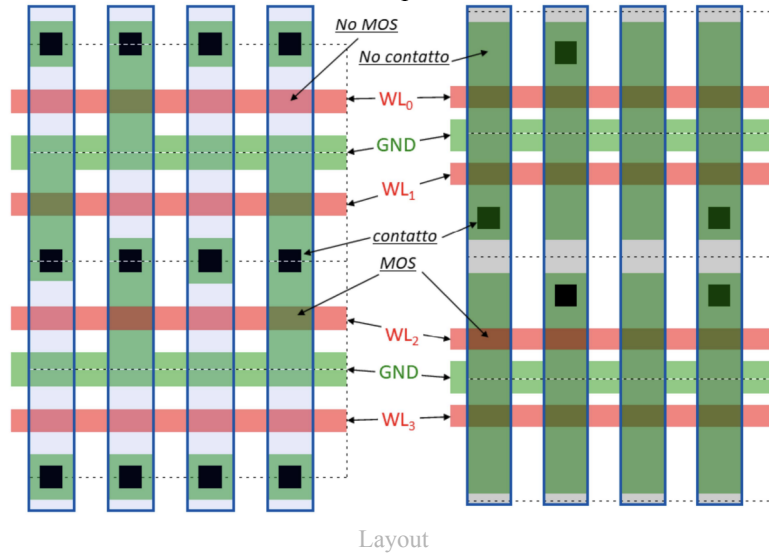
$$BL_1 = \overline{WL_0 + WL_2}$$

Il PMOS funge da **pull-up passivo**, infatti se nella bitline il bit è assente (1) si leggerà un $V_{DD} - V_{tp} = "1"$ ma se c'è l'NMOS questo riuscirà a tirare giù a $V_{tn} = "0"$ la bitline.

L'assenza del PMOS comprometterebbe la funzionalità del circuito in quanto V_{DD} rimarrebbe costante.

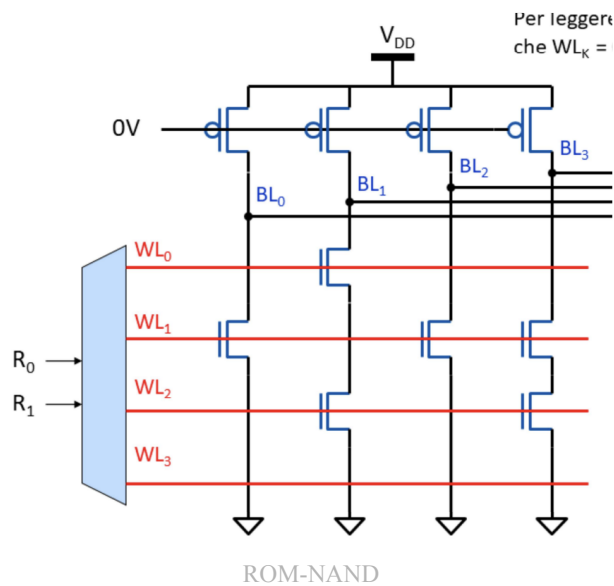
Ci sono due possibili processi di produzione:

- **Maschera Active:** tutti i mos presenti, ma i contatti vengono aggiunti in modo personalizzato
- **Maschera Contact:** ci sono tutti i contatti e solo i MOS per indicare i bit 0



NAND

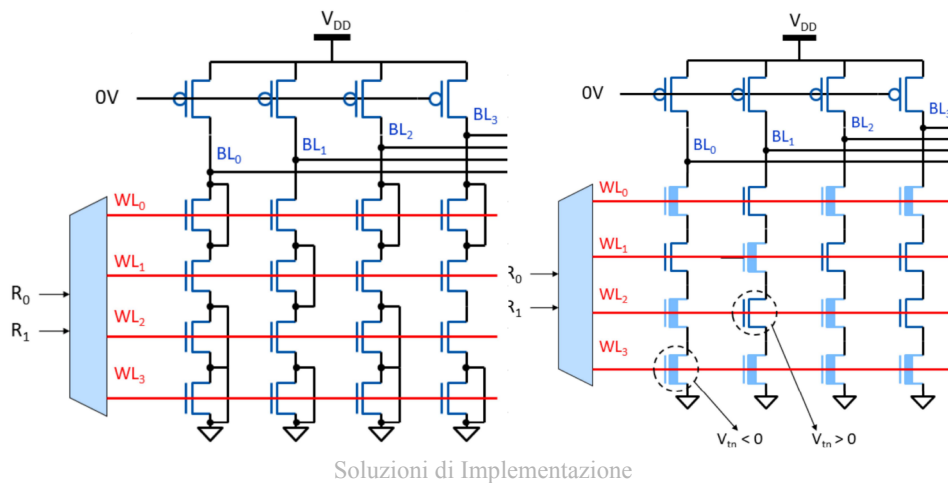
In questo caso il decoder sarà in logica negata. Il MUX non è presente in immagini ma si trova a dx dove vanno le 4 bitline



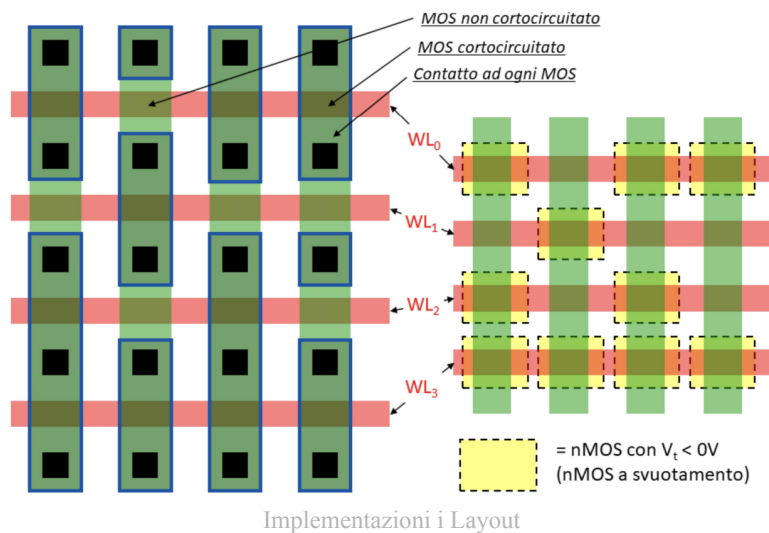
Il nome viene da come si scrivono le funzioni logica:

$$BL_1 = \overline{WL_0} \cdot WL_2$$

Le due soluzioni di implementazione sono le seguenti:



- La prima è detta METAL1: tutti i mos sono inseriti ma quelli da salvare come 0 sono in CC
- La seconda ACTIVE: qui tutti i mos sono inseriti ma gli zeri sono svuotamento che si comporta come un CC dal punto di vista logico ($v_t < 0$).

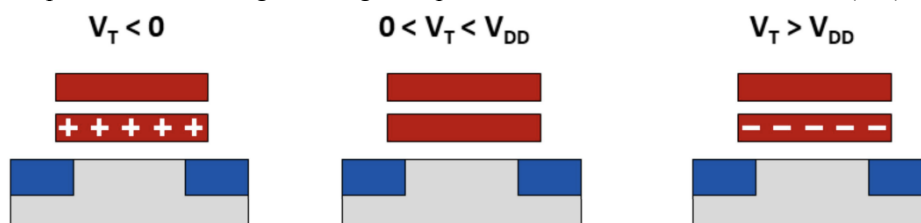


9.3) Memorie Riscrivibili

Per le memorie non volatili, l'informazione viene salvata come soglia di tensione dei MOS. Infatti un MOS con $V_{tn} > V_{DD}$ è sempre acceso, mentre uno con $V_{tn} < V_{DD}$ è sempre spento. Come facciamo per cambiarle?

MOSFET a gate flottante (FGMOS)

Questo MOS contiene due gate sovrapposti. Quello più vicino al MOS si chiama Gate Flottante (FG) che si andrà a caricare con carica positiva, nulla o negativa. il gate superiore è chiamato Gate di Controllo (CG)



Gate Flottante nelle 3 Configurazioni

Chiamando C_1, C_2 le capacità del CG e FG rispettivamente:

$$C_1(V_{FG} - V_{CG}) + C_2(V_{FG} - V_S) = 0 \implies V_{FG} = \frac{C_1 V_{CG} + C_2 V_S + Q}{C_1 + C_2}$$

Considerando $V_S = 0, V_{CG} = V_{GS}$ abbiamo

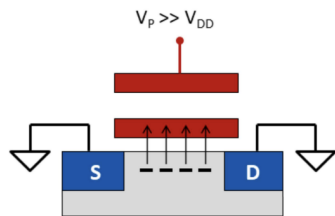
$$V_{FG} = \frac{C_1 V_{GS} + Q}{C_1 + C_2}$$

Definendo V_{ON} come la tensione FG necessaria per creare il canale:

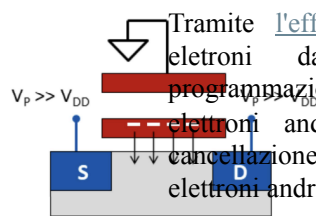
$$V_{GS} > \frac{V_{ON}(C_1 + C_2) - Q}{C_1} = V_{ON} \frac{C_1 + C_2}{C_1} - \frac{Q}{C_1} = V_T$$

Da qui è evidente il contributo della carica.

Ora che abbiamo capito il funzionamento non ci resta che trovare i meccanismi di cancellazione/programmazione del FGMOS



Programmazione
($V_t < 0 \rightarrow V_t > 0$)

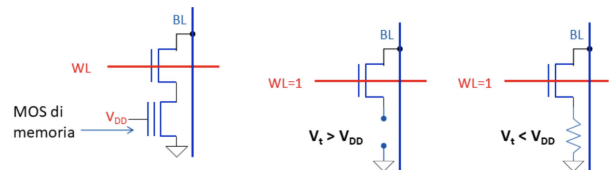


Cancellazione
($V_t > 0 \rightarrow V_t < 0$)

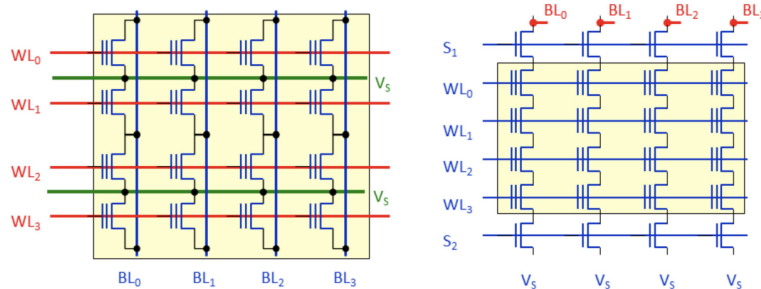
prog/Canc FGMOS

Applicazione in una memoria **E²PROM** (Electrically Erasable and Programmable ROM). In questo caso l'informazione è contenuta dai FGMOS mentre la WL è pilotata dal NMOS.

Per la scrittura si usa una combinazione di V_{BL} e V_{CL} adatti per eseguire la cancellazione o programmazione



EEPROM



EEPROM vs FLASH EPROM

Manca il pass-T di selezione, altrimenti si riscrive una linea per volta!

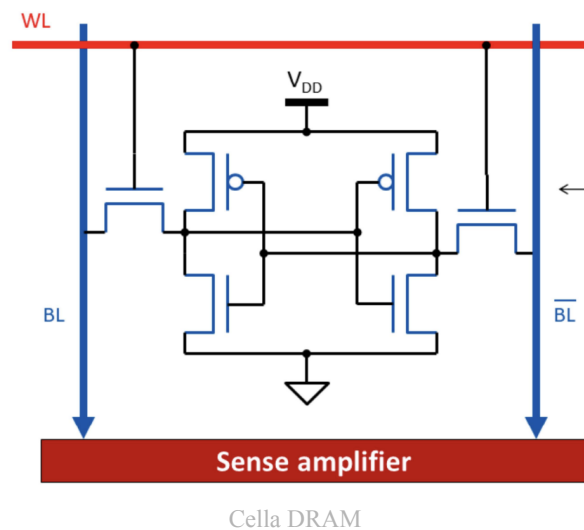
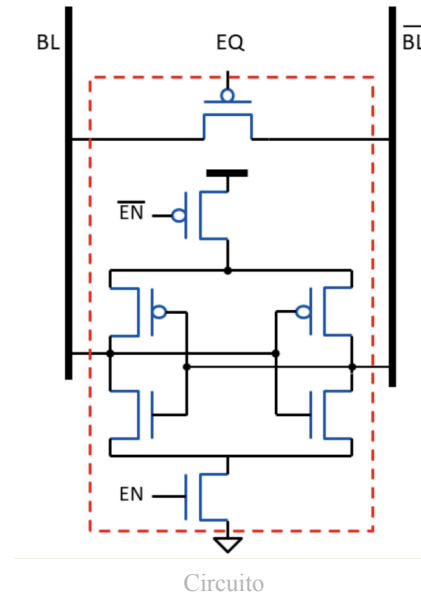
☑ non l'ho capito bene

9.4) RAM Statica (SRAM)

Una memoria SRAM è volatile ma ha il vantaggio di essere composta da solo 6 transistor (sense amplifier), WL e 2 BL ($BL, \overline{BL}$) per ogni bit d'informazione.

Sense Amplifier (SA)

Un SA è un a coppia di invertitori ad anello abilitati da EN. I segnale EQ invece serve a isolare le BL o a portarle a V_M



Condizione di riposo: Con $WL = 0$ BL mantiene il dato memorizzato per principio di bistabilità

Scrittura: Imponendo il valore desiderato nelle BL e abilitando $WL = 1$. se i pass-T sono abbastanza grandi il valore viene forzato su Q (e $\overline{Q}$). Finita la scrittura $WL=0$

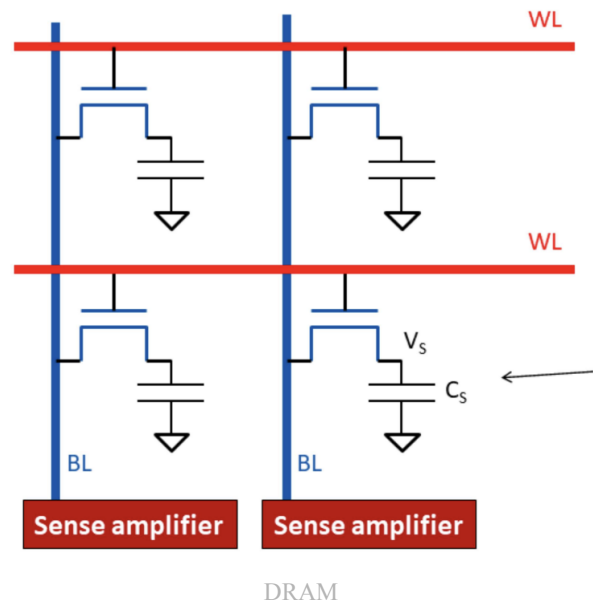
Lettura:

- La fase di lettura inizia con $EQ=1$ $EN=0$ $WL=0$.
- Per un pò viene tenuto $EN=1$, $EQ=0$ così le BL vengono caricate a V_M .
- Poi il SA viene disabilitato ($EN=0$) e le BL isolate ($EQ=1$)
- La WL viene attivata e dopo un tot di tempo la BL sarà al valore desiderato
- SA viene riattivato ($EN=1$)

9.5) Dynamic RAM (DRAM)

Questa architettura di memoria riesce a mettere un solo transistor + una capacità per ogni bit d'informazione, dove ogni BL necessita di un singolo SA. In questo caso si userà l'implementazione di un latch dinamico con una capacità molto grande (centinaia di fF).

Questa sarà allora volatile e più piccola ma anche più lenta.



Scrittura: La BL viene portata alla tensione desiderata, la WL è portata ad 1. La capacità si carica/scarica grazie al pass-T. La VL si spegne e il pass-T isola C.

Lettura:

Inizialmente il SA carica BL a una tensione media V_P e poi la isola. Poi la WL viene alzata e C_S si collega alla BL che infine avrà la tensione

$$V_{BL} = V_P + \frac{C_S}{C_{BL} + C_S}(V_P - V_S) \text{ la differenza di carica sarà: } \Delta V_{BL} = \frac{C_S}{C_{BL} + C_S}(V_P - V_S)$$

Il SA viene attivato e ci troviamo in 2 possibili casi:

- $V_S = 0$ quindi $V_{BL} < V_P$ e il SA scarica BL a 0
 - $V_S = 1$ quindi $V_{BL} > V_P$ e il sa carica BL a 1
- Infine WL viene abbassata

L'informazione quindi viene letta, distrutta e poi rigenerata. Inoltre dato che la variazione deve essere molto grande si cerca di costruire una capacità molto grande.

Se una cella non viene letta l'informazione viene comunque persa per la scarica di C. Quindi è necessario un ciclo di refresh in cui si legge tutta la memoria riga per riga.

Il ciclo di refresh è strutturato nel seguente modo:

- Precarica della BL (da 0 a V_P): $E = C_{BL}V_{DD}(V_P - 0)$
- Attivazione WL (da 0 a V_P): $E = C_{WL}V_{DD}(V_{DD} - 0)$
- Ripartizione carica da C_{ST} e C_{BL} : $E = 0$. Ora le cariche saranno $V_{SL} = 0$, $V_{SH} = V_{DD} - V_{tn}$. Da qui si trova $V_{BL} = \frac{C_{BL}V_P + C_{ST}V_S}{C_{ST} + C_{BL}}$
- O scarica di BL a 0: $E = 0$
- Oppure carica della BL a V_{DD} : $E = C_{BL}V_{DD}(V_{DD} - V_{BL}) + C_{ST}V_{DD}(V_{SH} - V_{BL})$. Notare come la capacità non si carichi fino a V_{DD} ma fino a V_{SH} .

Remark.

Le Cariche si bilanciano nel seguente modo:

- Prima dell'attivazione di WL

$$C_{BL}V_P + C_{ST}V_S$$

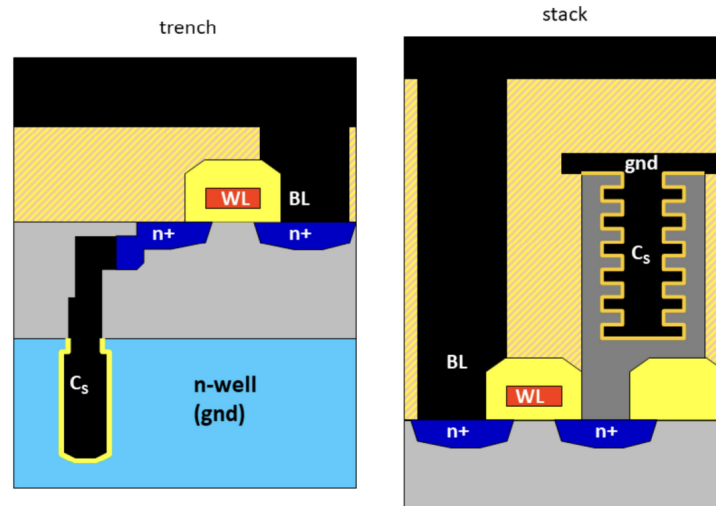
- Dopo l'attivazione di WL

$$(C_{BL} + C_{ST})V_{BL}$$

Per la conservazione delle energie queste si devono uguagliare

Mettendo il SA al centro della WL TODO

Le due possibili implementazioni sono:

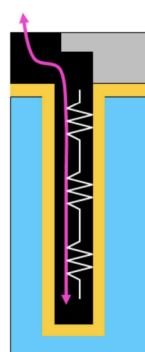


DRAM

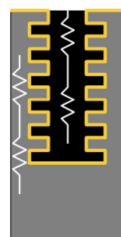
Nello stack la struttura della capacità permette di massimizzarne l'area. Tuttavia la resistenza dei connettori può dare problemi di resistenza se l'altezza della capacità è troppo alta

Accesso alla capacità di storage:

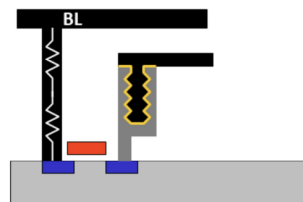
La trench deve essere molto sottile per aumentare la densità. Il limite è dato dalla resistenza serie



Lo stesso problema si presenta per la struttura stack



La struttura stack soffre anche di una elevata resistenza di accesso alla bitline a causa della profondità del contatto



DRAM

☑ buffer bidirezionale

10) Recap

- Recap dei modelli a canale corto dei n/p MOSFET

- NMOS**

Spento se $V_{GS} < V_{tn}$

Quando è acceso ci sono 3 zone che si scelgono in base al $\min\{V_{DS}, V_{GS} - V_{TN}, V_{DSATN}\}$

- V_{DS} : Regione Lineare
- $V_{GS} - V_{TN}$: Regione Saturazione (Pinch Off) $V_{DS} = V_{GS} - V_{TN}$
- V_{DSATN} : Regione Saturazione Velocità

$$I_{DS} = k'_n \frac{W_n}{L_n} \left(V_{GS} - V_{TN} - \frac{V_{MIN}}{2} \right) V_{MIN} \cdot (1 + \lambda_n V_{DS})$$

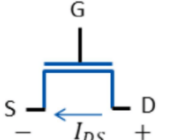
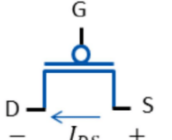
- PMOS**

Spento se $V_{GS} > V_{TP}$

Quando è acceso ci sono 3 zone che si scelgono in base al $\max\{V_{DS}, V_{GS} - V_{TP}, V_{DSATP}\}$

- V_{DS} : Regione Lineare
- $V_{GS} - V_{TP}$: Regione Saturazione (Pinch Off) $V_{DS} = V_{GS} - V_{TP}$
- V_{DSATP} : Regione Saturazione Velocità

$$I_{DS} = k'_p \frac{W_p}{L_p} \left(V_{GS} - V_{TP} - \frac{V_{MAX}}{2} \right) V_{MAX} \cdot (1 + \lambda_p V_{DS})$$

Convenzione dei segni di tensioni e correnti			
	substrato connesso al potenziale più basso (tipicamente la massa)		substrato connesso al potenziale più alto (tipicamente l'alimentazione)
$V_{DS} > 0$	$V_{SB} \geq 0$	$V_{DS} < 0$	$V_{BS} \geq 0$
Convenzione dei segni dei parametri			
$k'_n > 0$	$\lambda_n > 0$	$V_{DSATN} > 0$	$\gamma_n > 0$
$\Phi_n > 0$		$k'_p > 0$	$\lambda_p < 0$
		$V_{DSATP} < 0$	$\gamma_p > 0$
		$\Phi_p > 0$	

Recappone

- Effetto body:**

$$V_{tn} = V_{tn0} + \gamma(\sqrt{V_{SB} + 2\phi_n} - \sqrt{2\phi_n})$$

$$V_{tp} = V_{tp0} - \gamma(\sqrt{V_{BS} + 2\phi_n} - \sqrt{2\phi_n})$$

Remark (Come Calcolare Iterativamente V_S).

- NMOS**

- $V_{FH} = V_G - V_{tn0}$ (recall $V_S = V_{FH}$ nei NMOS, usually $V_G = V_{DD}$)
- $V_{tn} \rightarrow V_{tn0} + \gamma(\sqrt{2\phi_n + V_{FH}} - \sqrt{2\phi_n}) \Rightarrow V_{FH} \rightarrow V_G - V_{tn}$ ripetere fino a convergenza (recall usually $V_B = 0$)

- PMOS**

- $V_{FL} = V_G - V_{tp0}$ (recall $V_S = V_{FL}$ nei PMOS, usually $V_G = 0$)
- $V_{tp} \rightarrow V_{tp0} - \gamma(\sqrt{2\phi_n + V_B - V_{FL}} - \sqrt{2\phi_n}) \Rightarrow V_{FL} \rightarrow V_G - V_{tp}$ ripetere fino a convergenza (recall usually $V_B = V_{DD}$)

Soglia di commutazione

$$V_M = \frac{V_{TN} + \frac{V_{DSATN}}{2} + r(V_{DD} + V_{TP} + \frac{V_{DSATP}}{2})}{1 + r}$$

con

$$r = -\frac{k'_p Z_p V_{DSATP}}{k'_n Z_n V_{DSATN}}$$

Consumo:

L'energia assorbita per compiere un ciclo (in realtà si assorbe in LH e si dissipa in HL) è dato da

$$E = C_{out(tot)} \cdot V_{DD}(V_H - V_L)$$

Quella immagazzinata (in LH) è

$$E = \frac{1}{2} C(V_H^2 - V_L^2)$$

Le energia si possono sommare, inoltre, data una frequenza fissa f , la potenza dinamica è data da

$$P_{dyn} = C_{out(tot)} V_{DD}(V_H - V_L) f$$

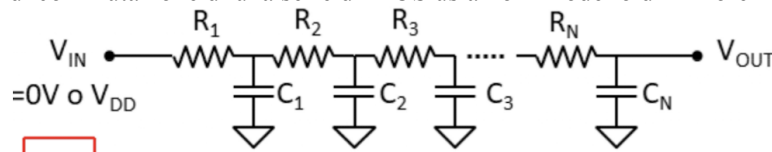
Il consumo dinamico in un circuito a più stadi (più PL in sequenza) è la somma di tutti i consumi dinamici

Funzione logica	P_F	Fattore di attività
AND	$P_A P_B$	$P_A P_B \cdot (1 - P_A P_B)$
NAND	$1 - P_A P_B$	$P_A P_B \cdot (1 - P_A P_B)$
OR	$p_A + (1 - p_A) p_B$	$[p_A + (1 - p_A) p_B] \cdot [(1 - p_A)(1 - p_B)]$
NOR	$(1 - p_A)(1 - p_B)$	$[p_A + (1 - p_A) p_B] \cdot [(1 - p_A)(1 - p_B)]$
XOR	$(p_A + p_B - 2p_A p_B)$	$(p_A + p_B - 2p_A p_B) \cdot (1 - p_A - p_B + 2p_A p_B)$
XNOR	$1 - p_A - p_B + 2p_A p_B$	$(p_A + p_B - 2p_A p_B) \cdot (1 - p_A - p_B + 2p_A p_B)$

Fattori di Attività Notevoli

-Elmore:

Per calcolare il tempo di commutazione di una serie di MOS usiamo il modello di Elmore



Modello RC MOS in Serie

$$t_p = 0.69 \sum_{i=1}^N \left(C_i \sum_{j=1}^i R_j \right)$$

Con R_n e R_p costanti è importante notare che una serie di N corrisponderà ad avere un parallelo di N nella rete opposta, quindi possiamo fare le seguenti supposizioni:

- Circuiti Sequenziali**

$t_{cont} = \min(t_p)$ e similmente $t_{prop} = \max(t_p)$. Quindi $T_{min} = t_{prop} + t_{setup} \rightarrow f_{max} = 1/T_{min}$. Inoltre

$t_{hold} < t_{cont}$

La capacità dinamica data dal CLK è $P_{dyn,clk} = V_{DD}^2 f N C_{in}$ con N il numero di FF

Ogni FF ha anche lui una potenza dinamica